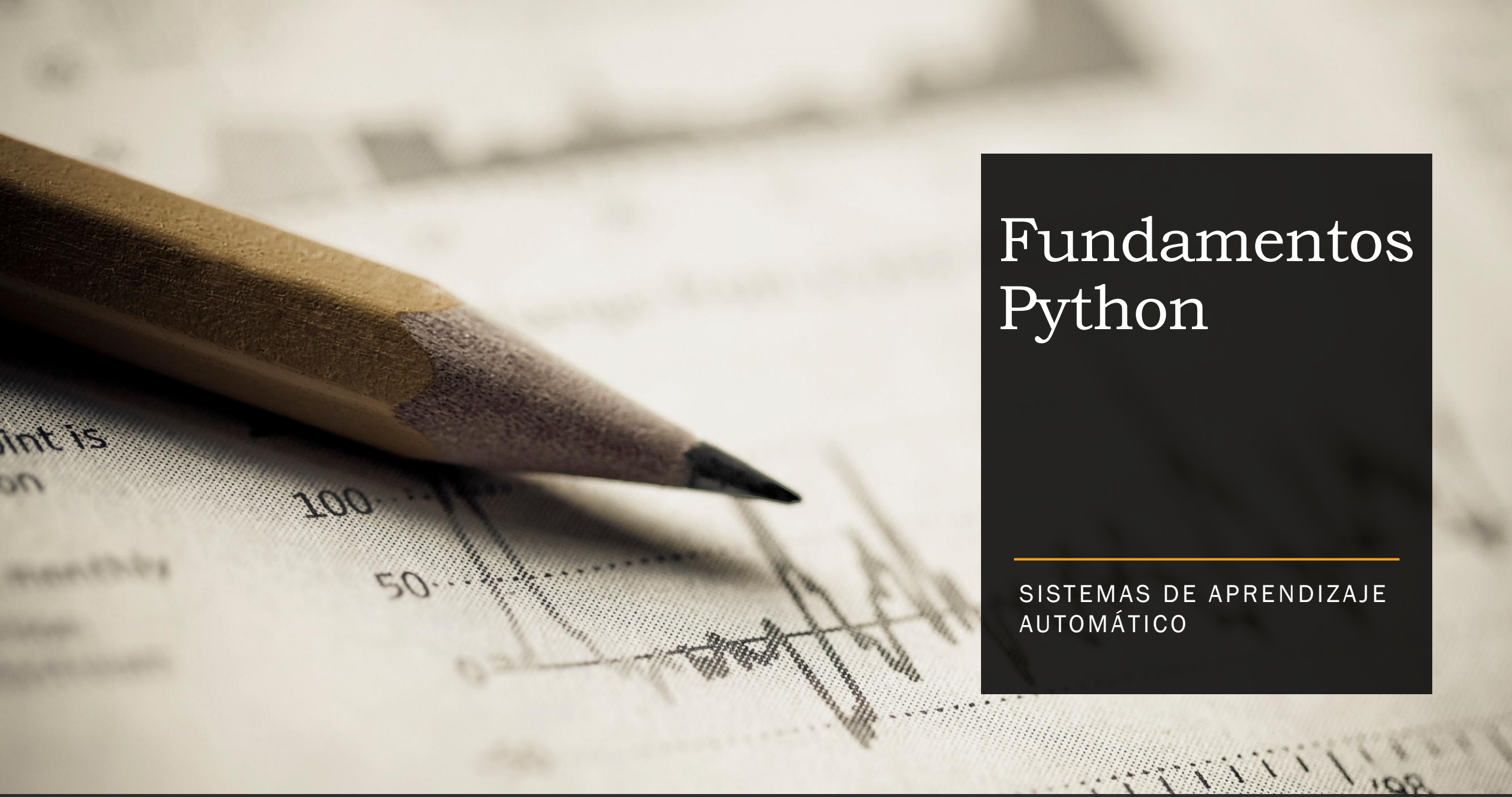


Fundamentos Python

SISTEMAS DE APRENDIZAJE AUTOMÁTICO

1. EL LEMA DE PYTHON

- Explícito es mejor que implícito.
- Hermoso es mejor que feo.
- Simple es mejor que complejo.
- Complejo es mejor que complicado.
- Plano es mejor que anidado.
- Escaso es mejor que denso.
- Cuenta la legibilidad.

- Si la implementación es difícil de explicar, es una mala idea.
- Si la implementación es fácil de explicar, puede ser una buena idea.

2. LENGUAJES DE PROGRAMACIÓN INTERPRETADOS

Actualmente, empresas o grupos de trabajo que desarrollan lenguajes de programación están apostando por se lenguajes interpretados, como la plataforma .NET Framework, Java, Ruby, PHP, JavaScript, Python, ...

Un lenguaje de programación interpretado es aquel que el código fuente se ejecuta directamente, instrucción a instrucción.

3. VENTAJAS E INCONVENIENTES

Ventajas de los lenguajes interpretados:

- Multiplataforma, puede ser interpretado en varios sistemas operativos
- Portabilidad, el mismo programa puede llevarse a diferentes plataformas

La principales desventaja son:

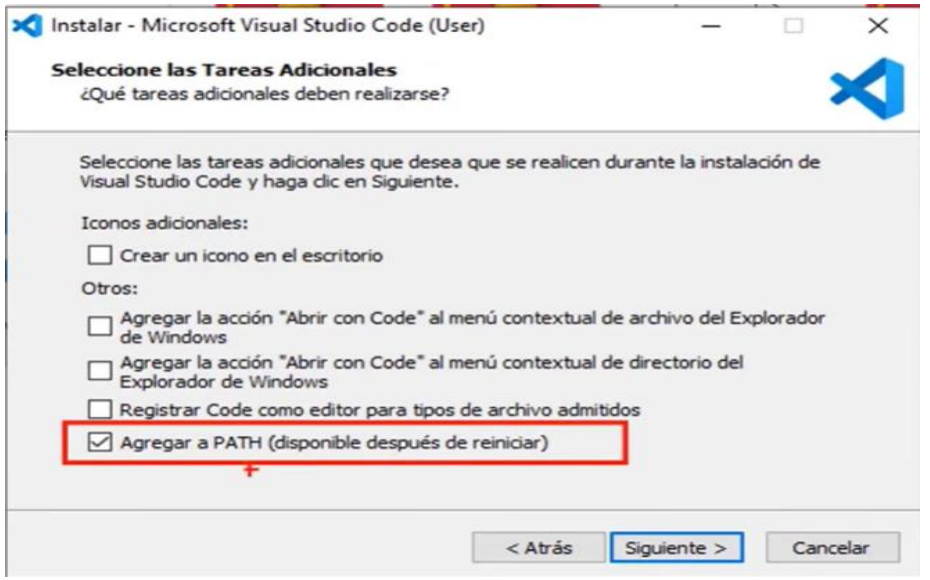
- Velocidad en la ejecución del programa, porque cuando compilamos lo transformamos a código máquina que es en el que funciona el procesador.
- Es necesario que en la máquina dónde va a funcionar tenga el interprete, ya sea como framework o como máquina virtual.

4. RESUMEN CARACTERÍSTICAS

- Creado por Guido Van Rossum a comienzos de los 90
- Lenguaje de alto nivel. Gramática sencilla, clara, muy legible.
- Tipado dinámico y fuerte.
- Orientado a objetos.
  - Sobrecarga de constructores. Herencia múltiple. Encapsulación. Interfaces. Polimorfismo
- Open Source.
- Fácil de aprender.
- Librería estándar muy amplia.
- Interpretado.
- Versátil.

5. INSTALACIÓN DEL ENTORNO DE DESARROLLO



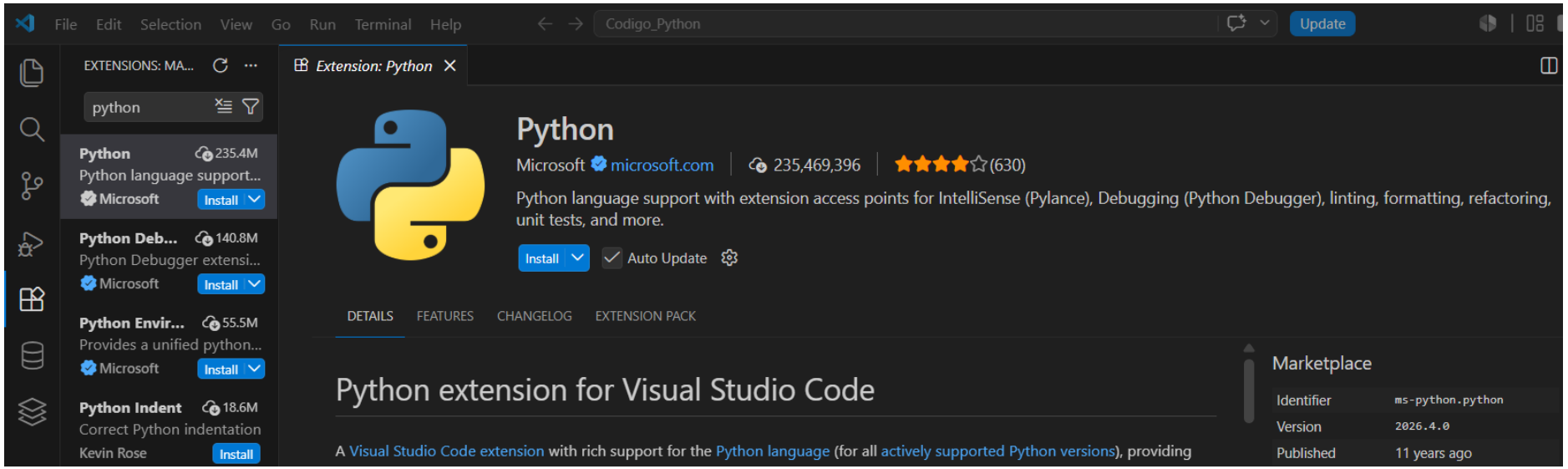


6. PRIMER PROGRAMA

Creación del Workspace (save workspace as ó open folder).

Instalación del plug in de Python.

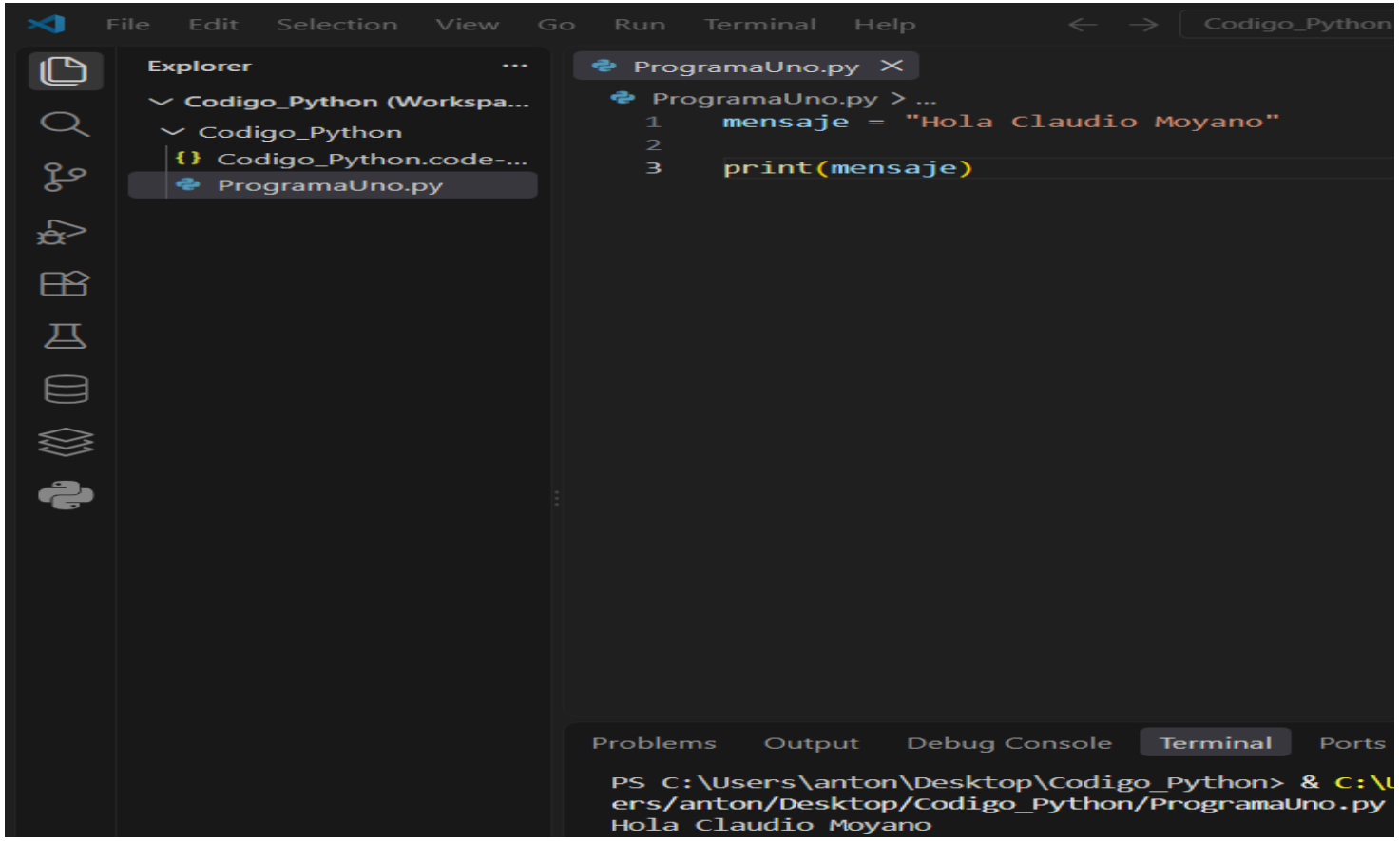
Nuevo archivo .py



Abrir el terminal (view terminal).

Run Python → run python file in terminal.

Cambiar apariencia del Visual Studio Code → File → Preferences → Settings.



7 SINTAXIS BÁSICA OPERADORES Y VARIABLES

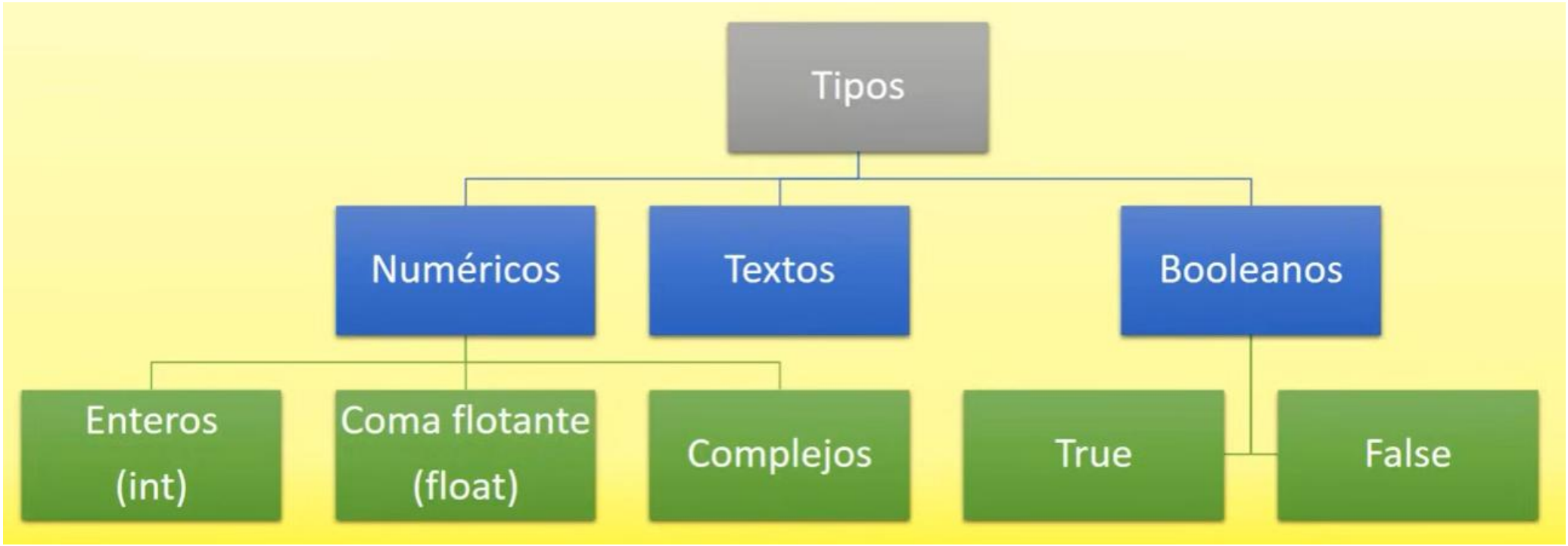


COMENTARIOS

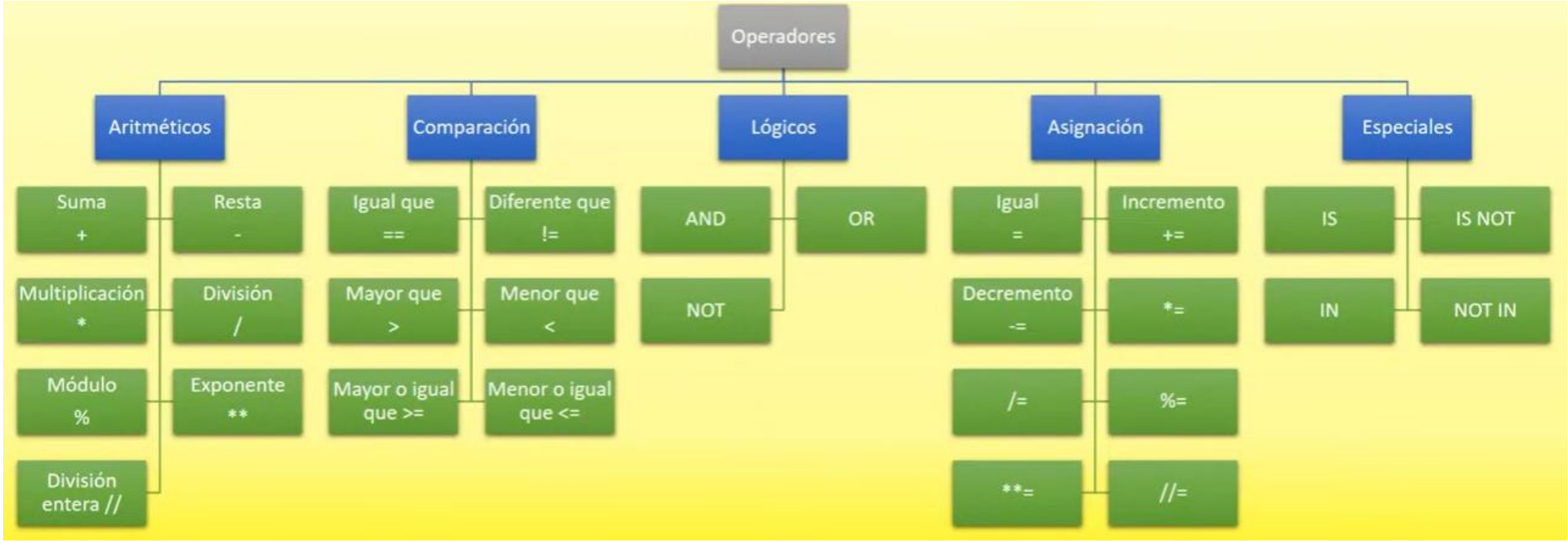
- # para comentar líneas de código
- """ """ para bloque de valores alfanuméricos que puede tener varias líneas

Avanzado

TIPOS ESENCIALES



OPERADORES ESENCIALES



VARIABLES

Declaración de Variables:

Python tiene tipado dinámico, asignando un tipo a los objetos en tiempo de ejecución en función de su valor, a diferencia del tipado estático donde las variables tienen un tipo.

Igualmente es un lenguaje fuertemente tipado, donde el tipo de los objetos no cambia repentinamente.

Un string que contiene solo dígitos no se convierte mágicamente en un número. Cada cambio de tipo requiere una conversión explícita.

EJEMPLO



DECLARACIÓN DE VARIABLES

- No especificamos tipo
- Un variable puede ser cualquier palabra a la que asignamos un valor
- El nombre de la variable siempre comienza por guion bajo o letra, nunca por número
- Diferencia mayúsculas de minúsculas
- **`print`** para mostrar el contenido de la variable en la pantalla
- **`type`** para mostrar el tipo de la variable
- **`del`** para eliminar una variable

EJEMPLO ASIGNACIÓN SIMULTANEA

Asignación Simultanea

Descubre la potencia de la asignación simultanea entre variables



non Avanzado

Intercambia los valores de las variables a y b para que a contenga 10 y b contenga 5.

EJEMPLO CONVERSIONES



CONVERSIONES

- **`str(n)`** convierte variables numéricas en texto
- **`int(s)`** convierte un texto en un valor numérico entero
- **`float(s)`** convierte un texto en un valor numérico con parte decimal

EJEMPLO CADENAS

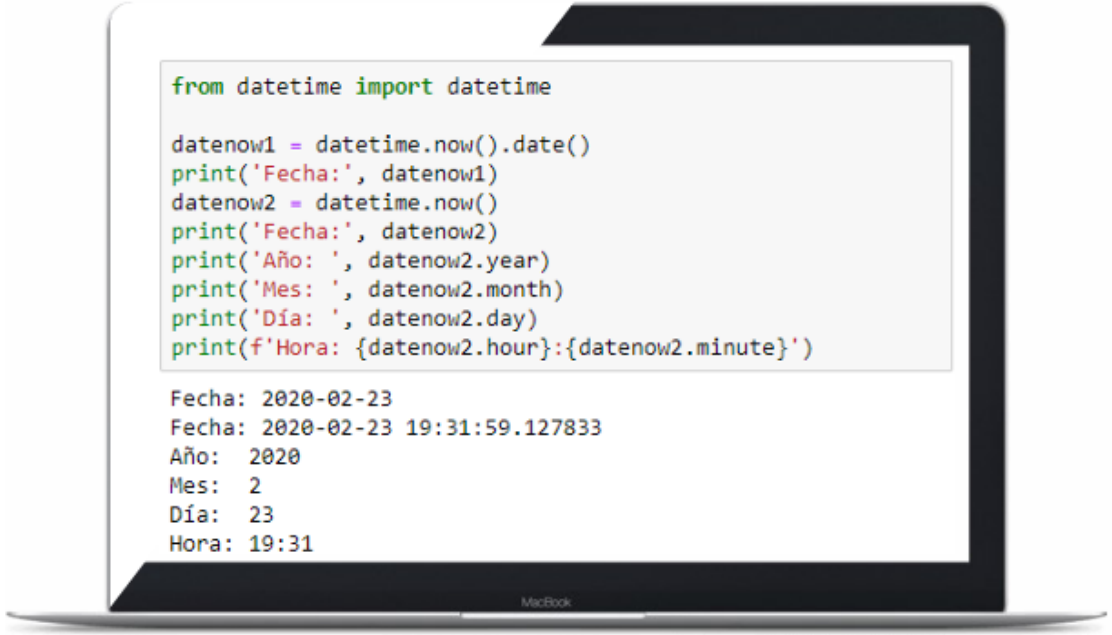


CADENA DE CARACTERES

- Las cadenas de caracteres son inmutables, no las podemos cambiar
- Si podemos cambiar el valor de una variables de tipo *String*
- Si podemos leer el contenido total o parcial de una cadena de caracteres
- **`len(s)`** retorna el número de caracteres de una cadena de texto
- Descubre las funciones de las cadenas pulsando punto después del nombre de la variable

Avanzado

EJEMPLO FECHAS Y HORAS

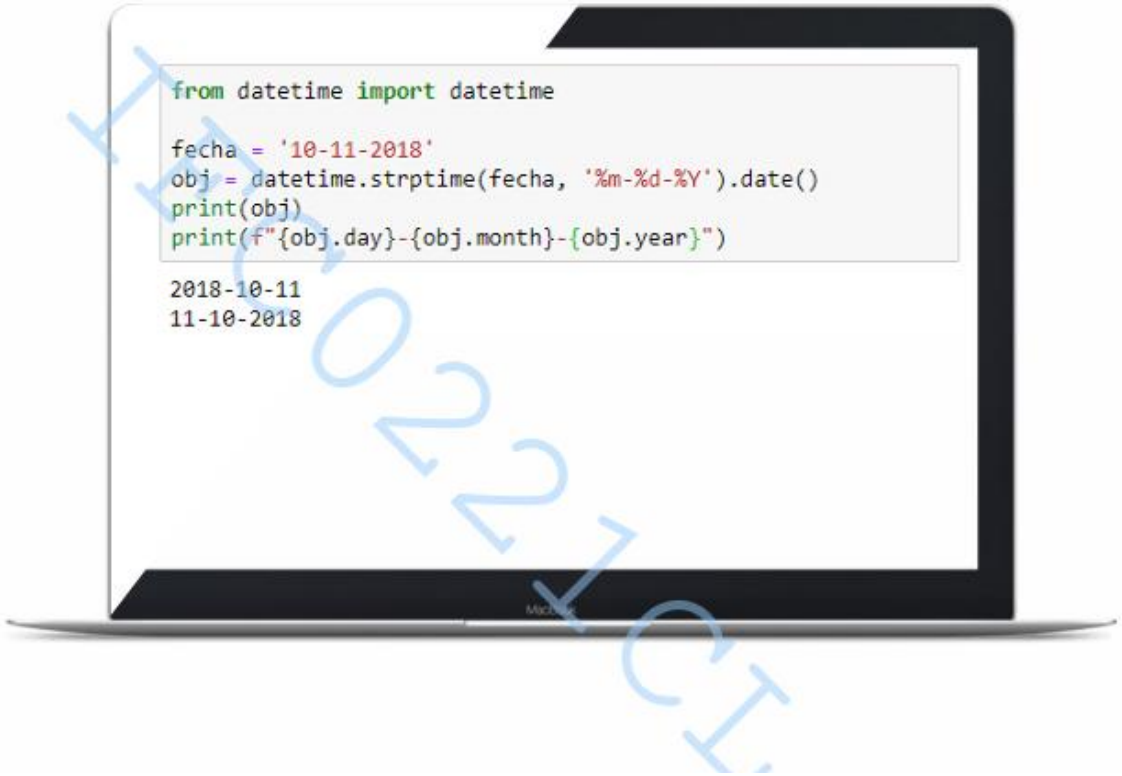


FECHAS Y HORAS

- Para trabajar con variables que almacenan fechas y horas tenemos que importar el módulo *datetime*
- **`now()`** retorna la fecha actual incluyendo la hora
- **`now().date()`** retorna la fecha actual

Avanzado

EJEMPLO PARSEAR FECHAS

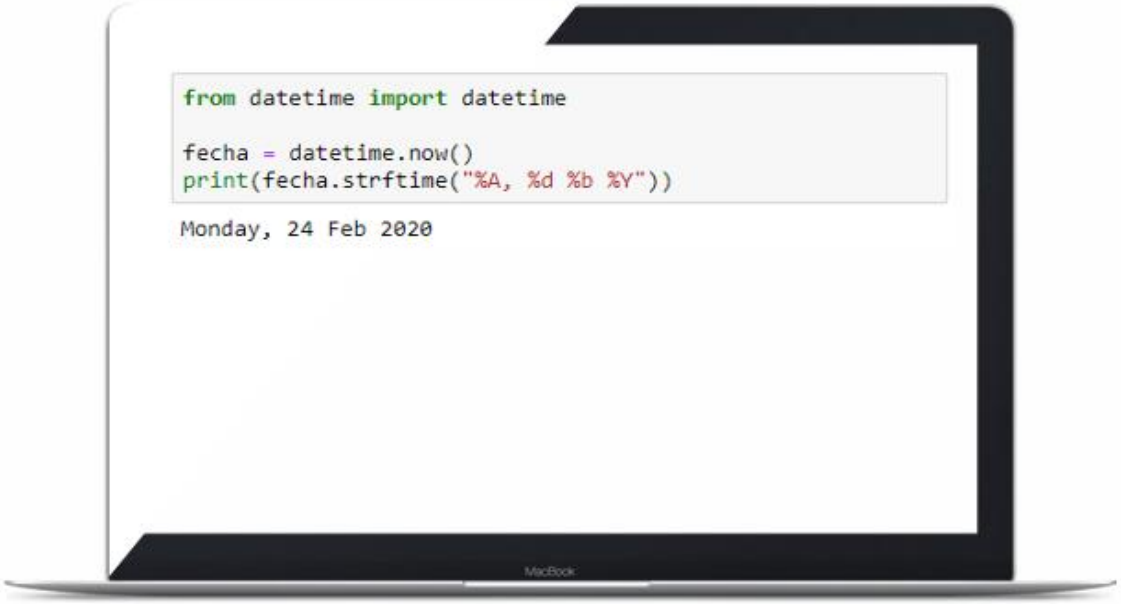


PARSE FECHAS

- **`strptime()`** convierte valores de texto en fecha



EJEMPLO FECHA FORMATO



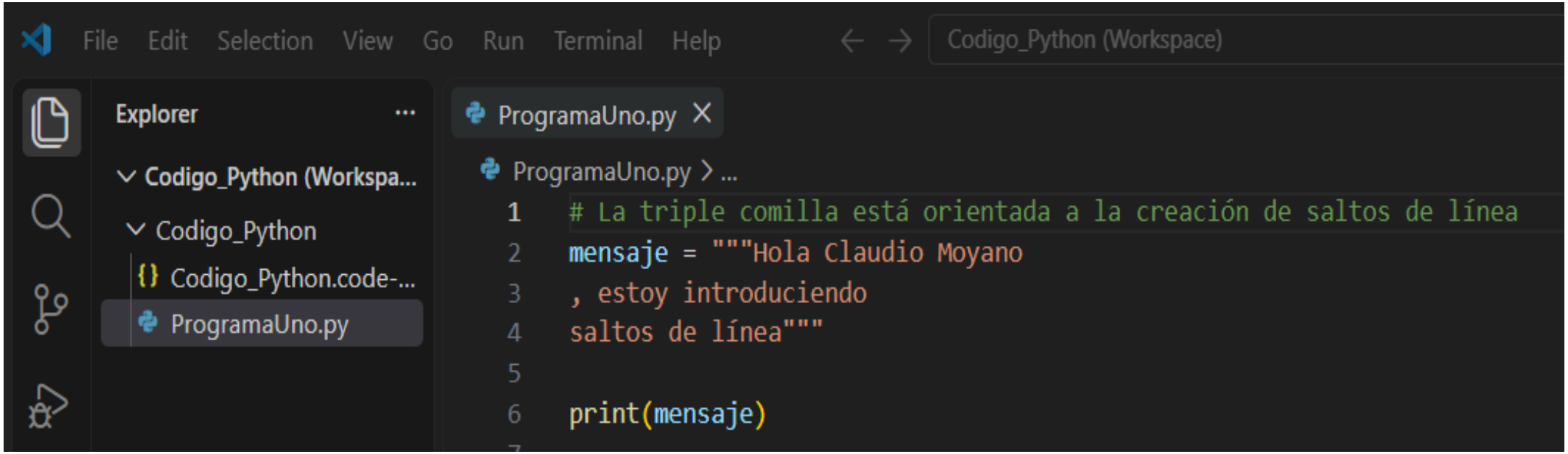
FORMAT FECHAS

- **strftime()** formatea la representación de fechas



Avanzado

EJEMPLO TRIPLE COMILLA



8. FUNCIONES

¿Qué son?

- Conjunto de líneas de código agrupadas (bloque de código) que funcionan como una unidad realizando una tarea específica.
- Las funciones en Python pueden devolver valores.
- Las funciones en Python pueden tener parámetros/argumentos
- A las funciones también se las denomina “métodos” cuando se encuentran definidas dentro de una clase

SINTAXIS

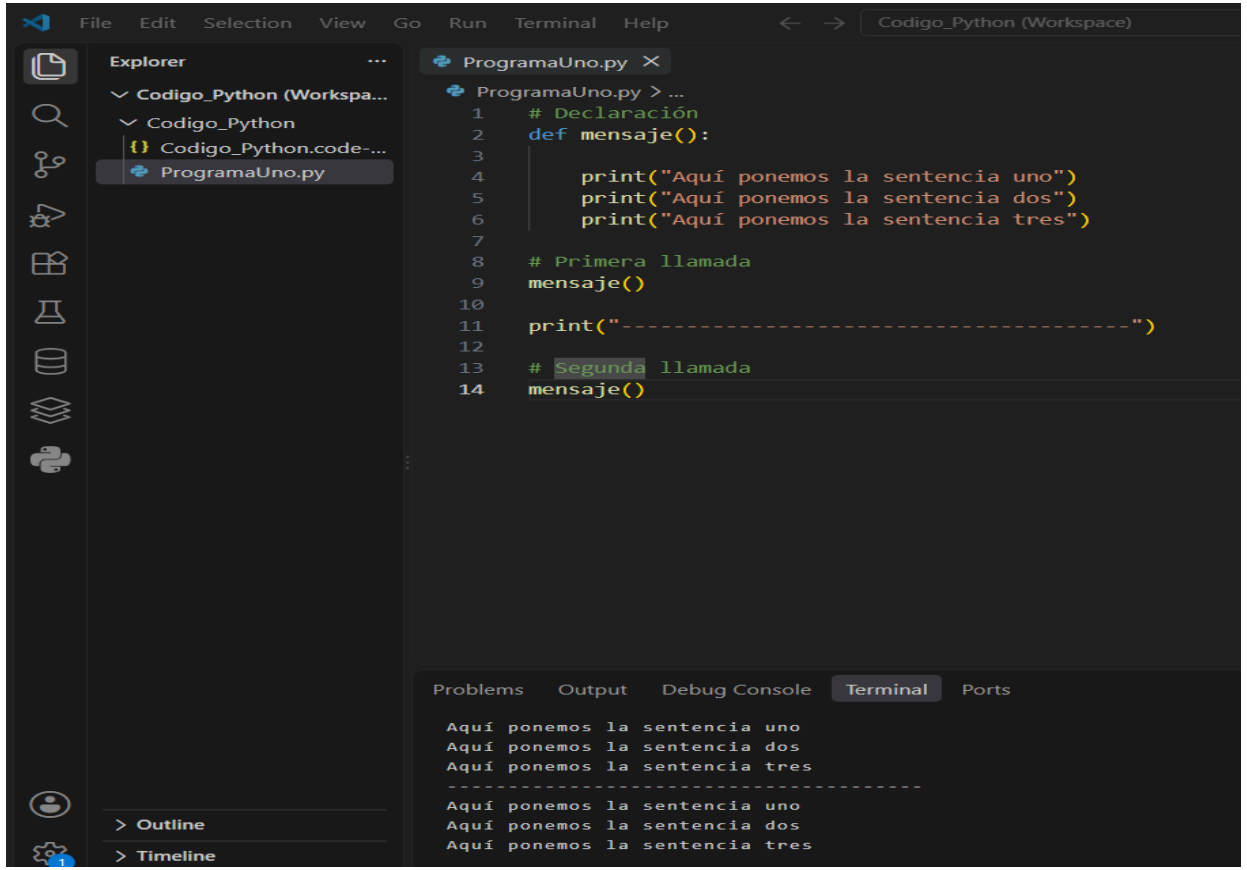
- def nombre_función():
  - Instrucciones de la función
  - return (opcional)
- def nombre_función(parámetros)
  - Instrucciones de la función
  - return (opcional)

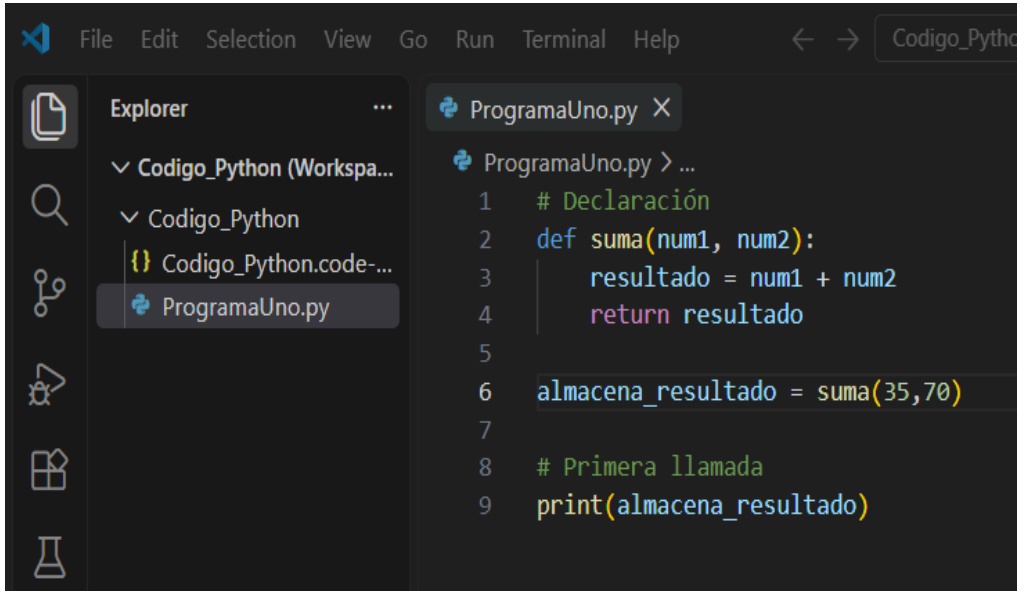
PALABRA RESERVADA DEF

Ejecución

- nombre_función()
- Nombre_función(parámetros)

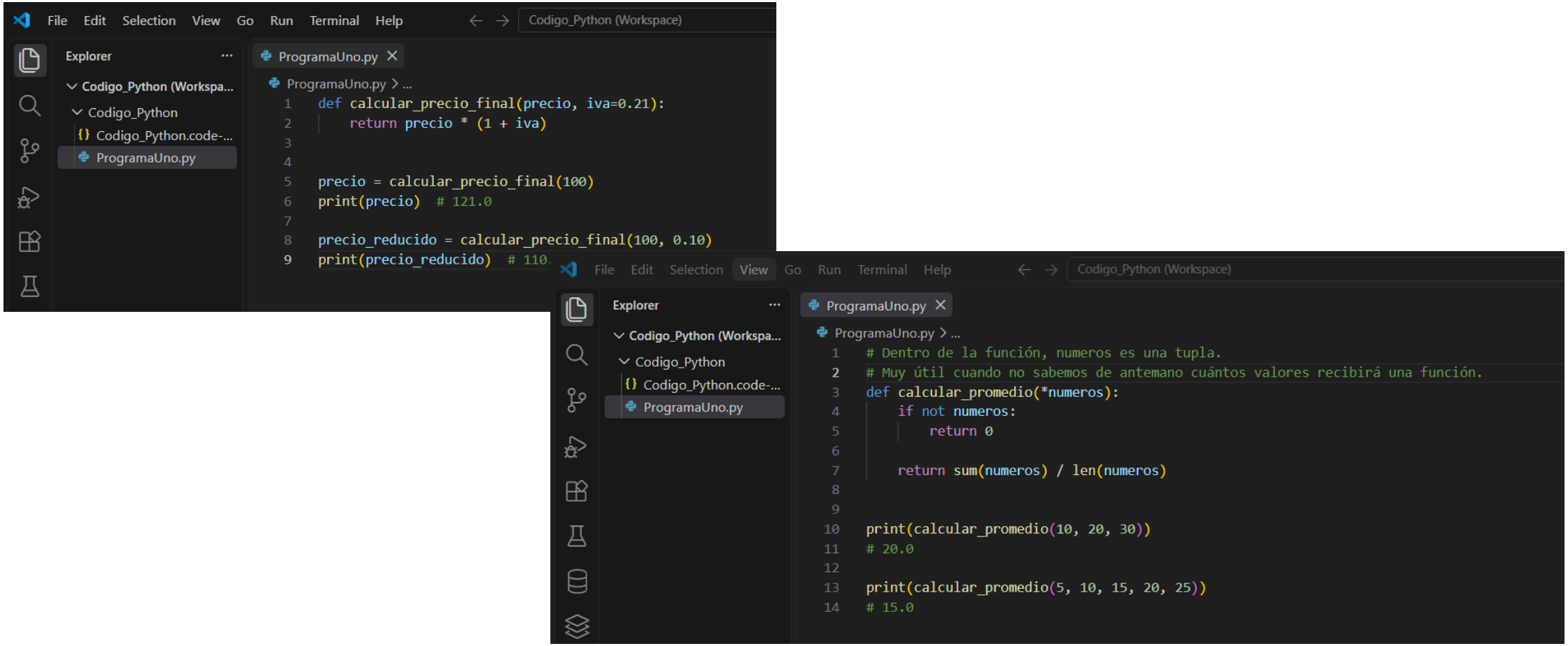
EJEMPLOS





Siempre por referencia.

EJEMPLOS


File Edit Selection View Go Run Terminal Help
Codigo_Python (Workspace)
Explorer
Codigo_Python (Workspa...
Codigo_Python
Codigo_Python.code-...
ProgramaUno.py
ProgramaUno.py
ProgramaUno.py > ...
def calcular_precio_final(precio, iva=0.21):
return precio * (1 + iva)
precio = calcular_precio_final(100)
print(precio) # 121.0
precio_reducido = calcular_precio_final(100, 0.10)
print(precio_reducido) # 110.
File Edit Selection View Go Run Terminal Help
Codigo_Python (Workspace)
Explorer
Codigo_Python (Workspa...
Codigo_Python
Codigo_Python.code-...
ProgramaUno.py
ProgramaUno.py
ProgramaUno.py > ...
Dentro de la función, numeros es una tupla.
Muy útil cuando no sabemos de antemano cuántos valores recibirá una función.
def calcular_promedio(*numeros):
if not numeros:
return 0
return sum(numeros) / len(numeros)
print(calcular_promedio(10, 20, 30))
20.0
print(calcular_promedio(5, 10, 15, 20, 25))
15.0

EJEMPLO



FUNCIONES

- Una *Función* es un bloque de código que se ejecuta cuando es llamado
- **`def`** es la palabra reservada de Python para crear una funciones
- **`return`** es la palabra reservada de Python para retornar un resultado desde una *Función*
- Las colecciones y otros objetos son parámetros por referencia

EJEMPLOS

```python
#Parámetros por posición
def resta(a, b):
    return a - b

resta(30, 10)
```

```
20
```

```python
#Parámetros por nombre
def resta(a, b):
    return a - b

resta(b=30, a=10)
```

```
-20
```

```python
#Parámetros por defecto
def resta(a=None, b=None):
    if(a == None or b == None):
        print("Error, debes enviar dos números a la función")
        return
    return a - b

resta(30, 10)
resta()
```

```
Error, debes enviar dos números a la función
```

```python
#Parámetros por indeterminados, args es una Tupla
def indeterminados(*args):
    for arg in args:
        print(arg)

indeterminados(5,"Hola Plone",[1,2,3,4,5])
```

```
5
Hola Plone
[1, 2, 3, 4, 5]
```

```python
#Parámetros por indeterminados, kwargs es una Diccionario
def indeterminados(**kwargs):
    print(kwargs)

indeterminados(n=5, c="Hola Plone", l=[1,2,3,4,5])
```

```
{'n': 5, 'c': 'Hola Plone', 'l': [1, 2, 3, 4, 5]}
```

```python
#Parámetros por indeterminados, kwargs es una Diccionario
def indeterminados(**kwargs):
    for kwarg in kwargs:
        print(kwarg, "=>", kwargs[kwarg])

indeterminados(n=5, c="Hola Plone", l=[1,2,3,4,5])
```

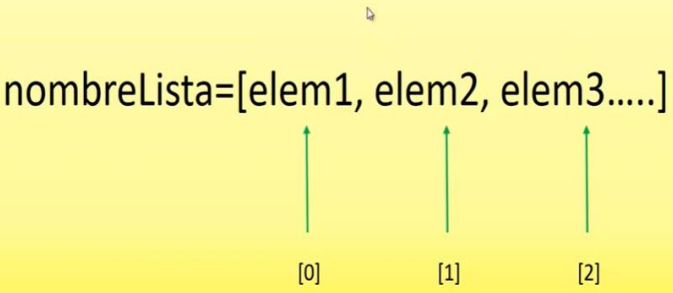
```
n => 5
c => Hola Plone
l => [1, 2, 3, 4, 5]
```

9. LISTAS

¿Qué son las listas?

- Estructura de datos que nos permite almacenar gran cantidad de valores (equivalente a los array en otros lenguajes de programación)
- En Python las listas pueden guardar diferente tipo de valores (en otros lenguajes no ocurre esto con los array)
- Se pueden expandir dinámicamente añadiendo nuevos elementos (otra novedad respecto a los arrays en otros lenguajes)

Sintaxis de las listas



EJEMPLOS

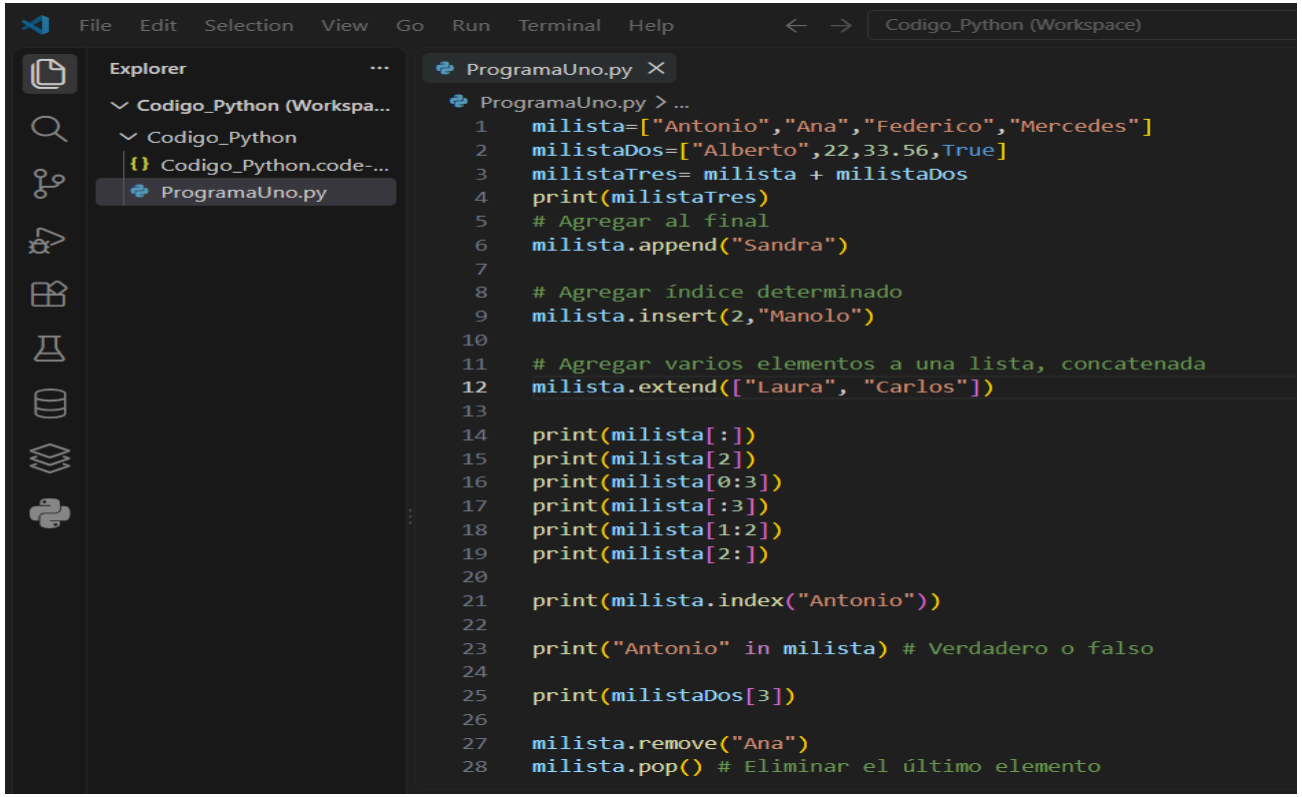


COLECCIONES, LISTAS

- Una *Lista* es una colección de elementos ordenados con un índice de base 0
- En una *Lista* se pueden añadir, eliminar y modificar elementos
- **`extend`** une dos Listas en una
- **`sort`** ordenar los elementos en base al valor
- **`reverse`** invierte el orden de los elementos en base a su índice
- **`pop(i)`** elimina el elemento de la posición i

Avanzado

EJEMPLOS


File Edit Selection View Go Run Terminal Help
Codigo_Python (Workspace)
Explorer
Codigo_Python (Workspa...
Codigo_Python
Codigo_Python.code-...
ProgramaUno.py
ProgramaUno.py
ProgramaUno.py > ...
1 milista=["Antonio","Ana","Federico","Mercedes"]
2 milistaDos=["Alberto",22,33.56,True]
3 milistaTres= milista + milistaDos
4 print(milistaTres)
5 # Agregar al final
6 milista.append("Sandra")
7
8 # Agregar índice determinado
9 milista.insert(2,"Manolo")
10
11 # Agregar varios elementos a una lista, concatenada
12 milista.extend(["Laura", "Carlos"])
13
14 print(milista[:])
15 print(milista[2])
16 print(milista[0:3])
17 print(milista[:3])
18 print(milista[1:2])
19 print(milista[2:])
20
21 print(milista.index("Antonio"))
22
23 print("Antonio" in milista) # Verdadero o falso
24
25 print(milistaDos[3])
26
27 milista.remove("Ana")
28 milista.pop() # Eliminar el último elemento

10. TUPLAS

¿Qué son las tuplas?

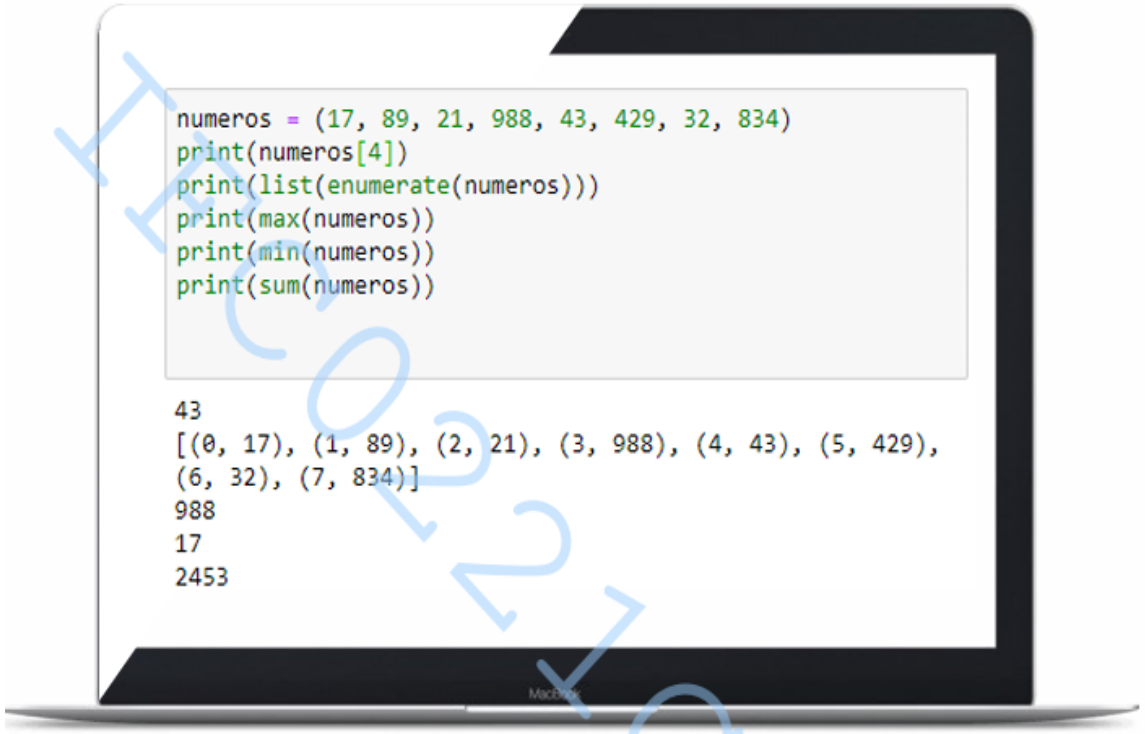
- Las tuplas son listas inmutables, es decir, no se pueden modificar después de su creación.
  - No permiten añadir, eliminar, mover elementos etc (no append, extend, remove)
  - Si permiten extraer porciones, pero el resultado de la extracción es una tupla nueva.
  - [redacted] permiten búsquedas [redacted]
  - Si permiten comprobar si un elemento se encuentra en la tupla.
- ¿Qué utilidad o ventaja tienen respecto a las listas?
  - Más rápidas
  - Menos espacio (mayor optimización)
  - Formatean Strings
  - Pueden utilizarse como claves en un diccionario. (las listas no)

Sintaxis de las tuplas

nombreLista=(elem1, elem2, elem3.....)



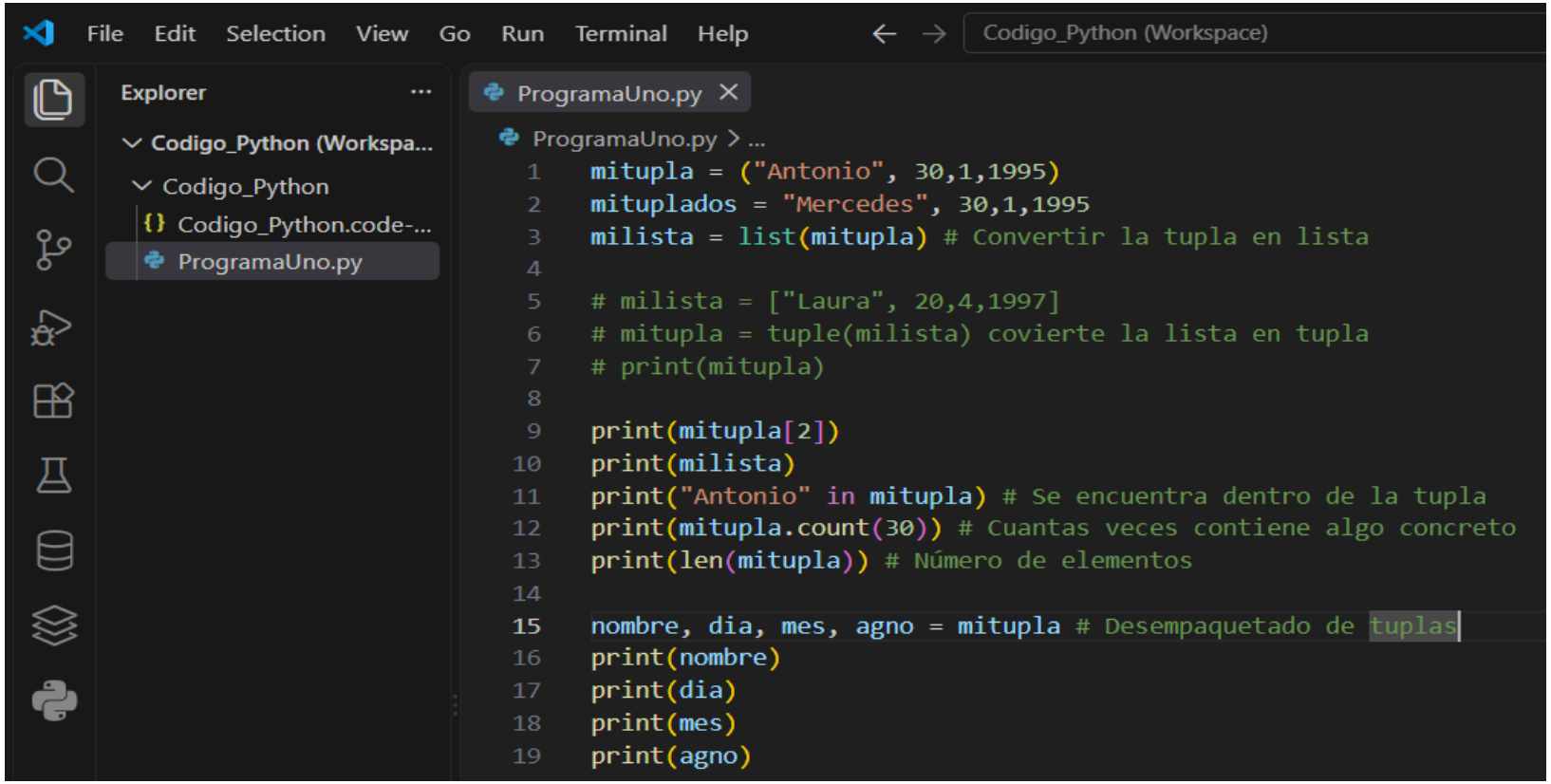
EJEMPLOS



COLECCIONES, TUPLAS

- Una *Tupla* es una colección de elementos ordenados con un índice de base 0
- En una *Tupla* NO se pueden añadir, eliminar y modificar elementos

EJEMPLOS


File Edit Selection View Go Run Terminal Help
Codigo_Python (Workspace)
Explorer
Codigo_Python (Workspa...
Codigo_Python
Codigo_Python.code-...
ProgramaUno.py
ProgramaUno.py
ProgramaUno.py > ...
1 mitupla = ("Antonio", 30,1,1995)
2 mituplados = "Mercedes", 30,1,1995
3 milista = list(mitupla) # Convertir la tupla en lista
4
5 # milista = ["Laura", 20,4,1997]
6 # mitupla = tuple(milista) covierte la lista en tupla
7 # print(mitupla)
8
9 print(mitupla[2])
10 print(milista)
11 print("Antonio" in mitupla) # Se encuentra dentro de la tupla
12 print(mitupla.count(30)) # Cuantas veces contiene algo concreto
13 print(len(mitupla)) # Número de elementos
14
15 nombre, dia, mes, agno = mitupla # Desempaquetado de tuplas
16 print(nombre)
17 print(dia)
18 print(mes)
19 print(agno)

11. DICCIONARIOS

¿Qué son los diccionarios?



- Estructura de datos que nos permite almacenar valores de diferente tipo (enteros, cadenas de texto, decimales) e incluso listas y otros diccionarios.
- La principal característica de los diccionarios es que los datos se almacenan asociados a una clave de tal forma que se crea una asociación de tipo **clave : valor** para cada elemento almacenado.
- Los elementos almacenados no están ordenados. El orden es indiferente a la hora de almacenar información en un diccionario.

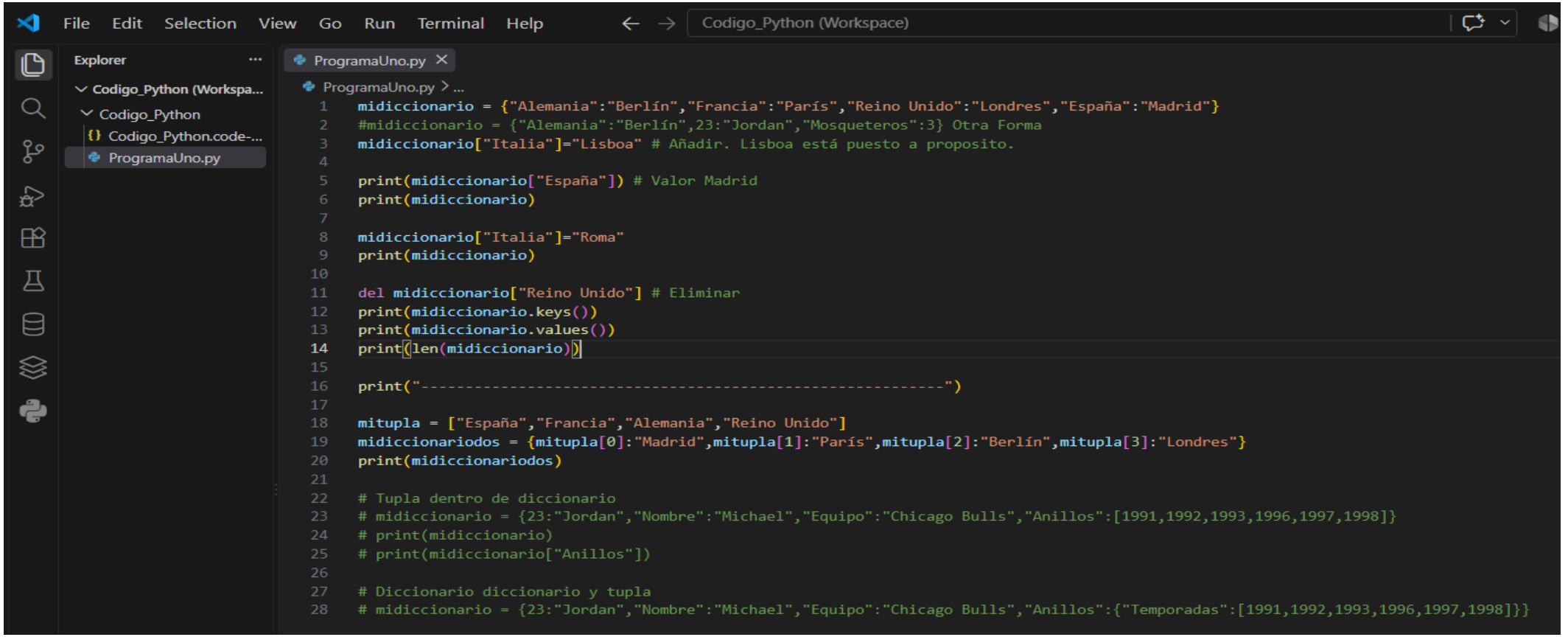
EJEMPLOS



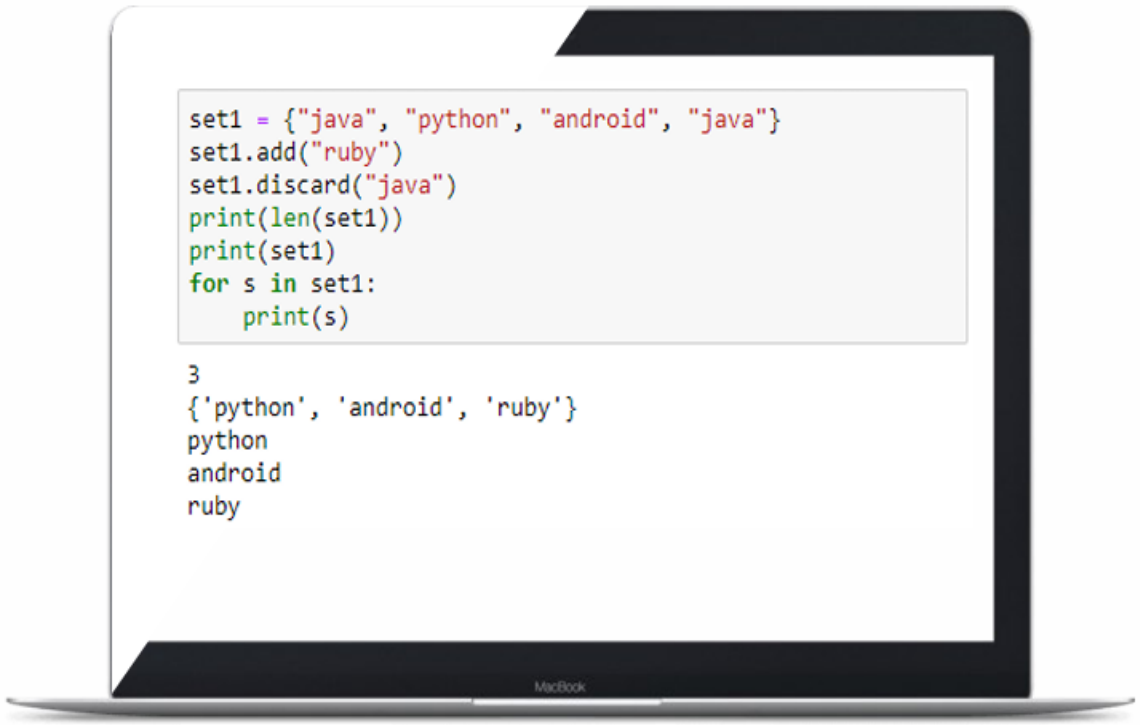
COLECCIONES, DICCIONARIOS

- Una *Diccionario* es una colección de elementos indexados por su clave
- Cada elemento del *Diccionario* se compone de clave y valor
- Cuando recorremos un *Diccionario* mediante *for* los valores que obtenemos son las claves
- **`get(k, "")`** muestra el valor de una clave o un valor alternativo si la clave no existe

EJEMPLOS


File Edit Selection View Go Run Terminal Help
Codigo_Python (Workspace)
Explorer
Codigo_Python (Workspa...
Codigo_Python
Codigo_Python.code-...
ProgramaUno.py
ProgramaUno.py
ProgramaUno.py > ...
1 midiccionario = {"Alemania":"Berlín","Francia":"París","Reino Unido":"Londres","España":"Madrid"}
2 #midiccionario = {"Alemania":"Berlín",23:"Jordan","Mosqueteros":3} Otra Forma
3 midiccionario["Italia"]="Lisboa" # Añadir. Lisboa está puesto a proposito.
4
5 print(midiccionario["España"]) # Valor Madrid
6 print(midiccionario)
7
8 midiccionario["Italia"]="Roma"
9 print(midiccionario)
10
11 del midiccionario["Reino Unido"] # Eliminar
12 print(midiccionario.keys())
13 print(midiccionario.values())
14 print(len(midiccionario))
15
16 print("--")
17
18 mitupla = ["España","Francia","Alemania","Reino Unido"]
19 midiccionariodos = {mitupla[0]:"Madrid",mitupla[1]:"París",mitupla[2]:"Berlín",mitupla[3]:"Londres"}
20 print(midiccionariodos)
21
22 # Tupla dentro de diccionario
23 # midiccionario = {23:"Jordan","Nombre":"Michael","Equipo":"Chicago Bulls","Anillos":[1991,1992,1993,1996,1997,1998]}
24 # print(midiccionario)
25 # print(midiccionario["Anillos"])
26
27 # Diccionario diccionario y tupla
28 # midiccionario = {23:"Jordan","Nombre":"Michael","Equipo":"Chicago Bulls","Anillos":{"Temporadas":[1991,1992,1993,1996,1997,1998]}}

12. CONJUNTOS

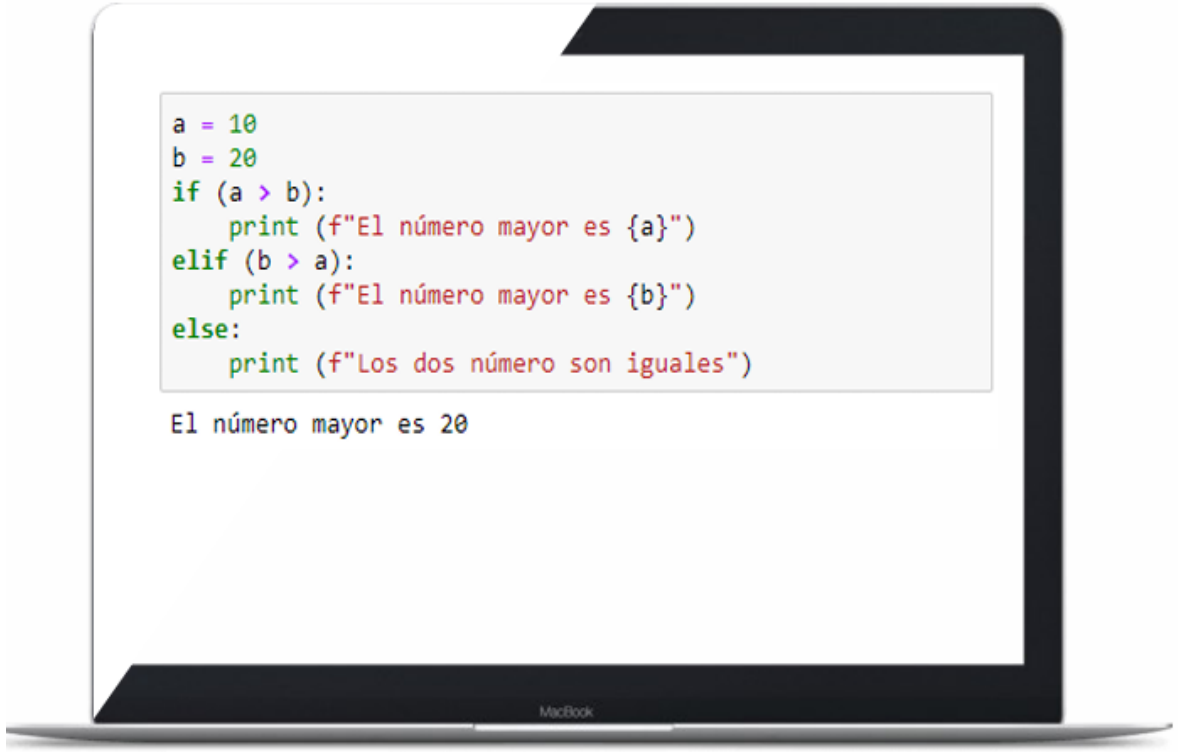


COLECCIONES, CONJUNTOS

- Un *Conjunto* es una colección de elementos sin índice, se dice que esta desordenado
- En un *Conjunto* se pueden añadir y eliminar elementos.
- Para acceder a los valores tenemos que recorrer la colección mediante un *for*

Avanzado

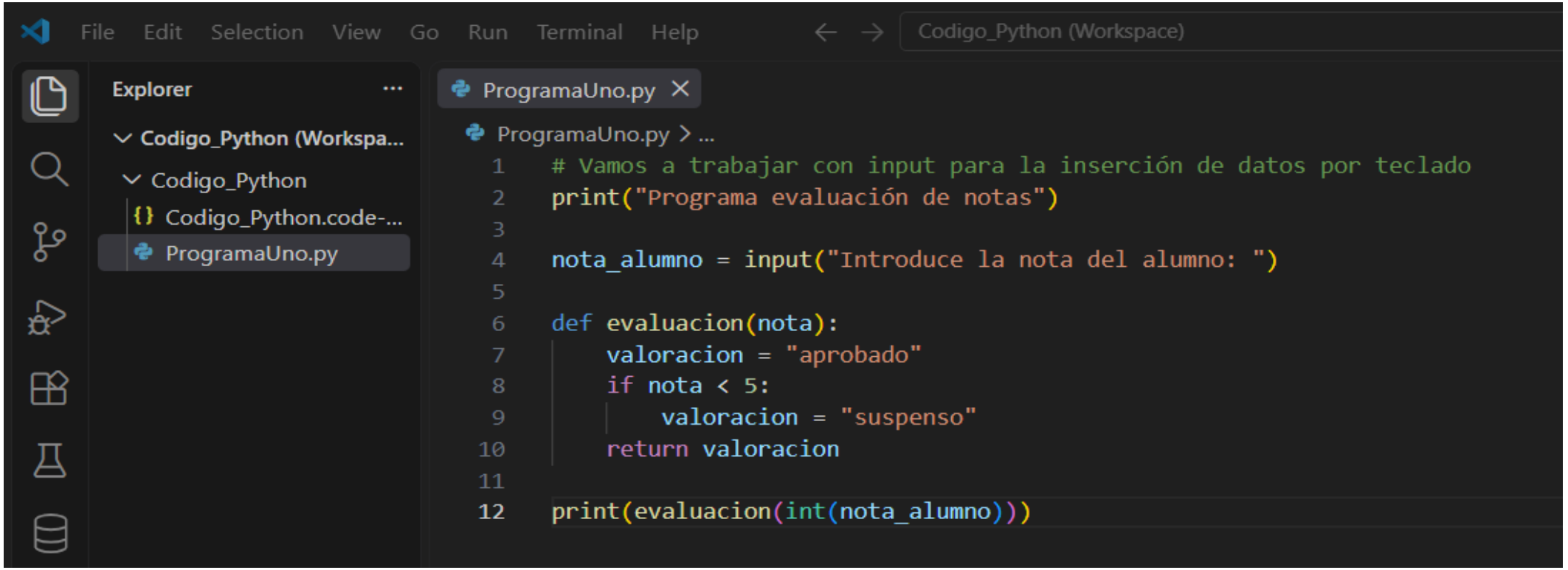
13. CONDICIONALES



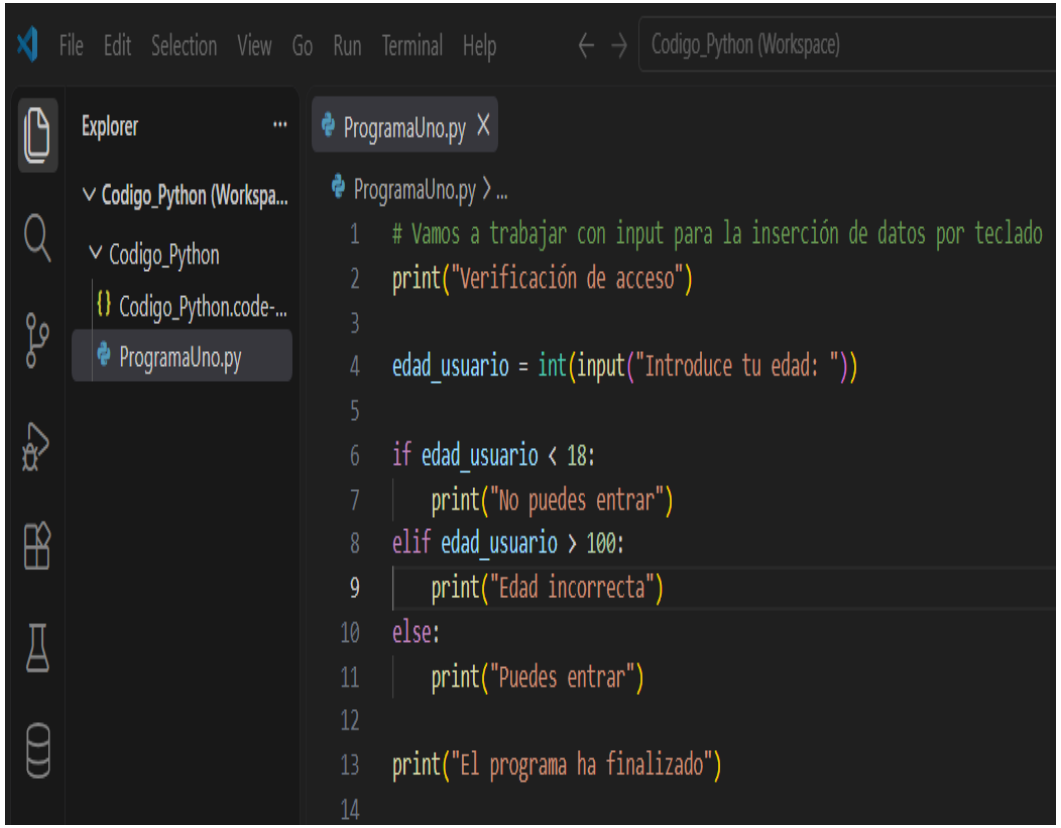
SENTENCIA DE DECISIÓN IF/ELSE

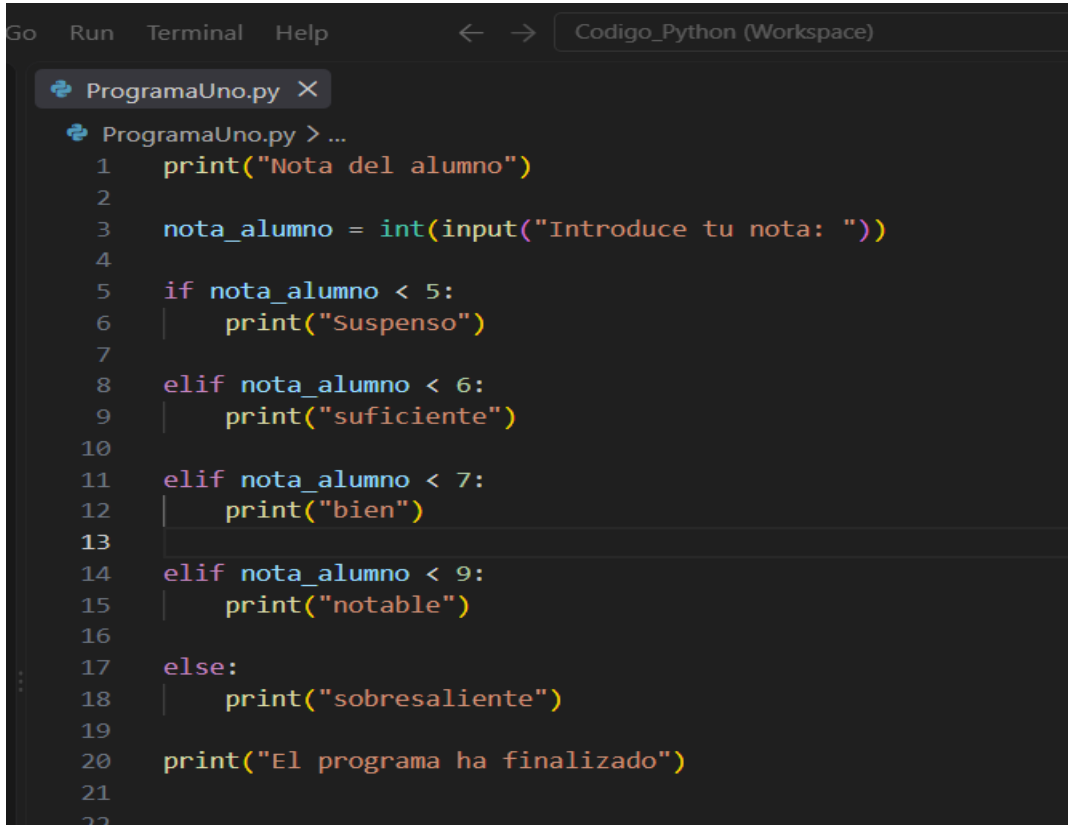
- Las sentencias de decisión determinar el flujo del programa tras evaluar una expresión de comparación
- **`if`** determina la condición y el bloque de sentencias que cumple la condición
- **`else`** marca el bloque de sentencias que no cumple la condición
- **`elif`** simplicamos un *else if* (entonces si)

EJEMPLO IF

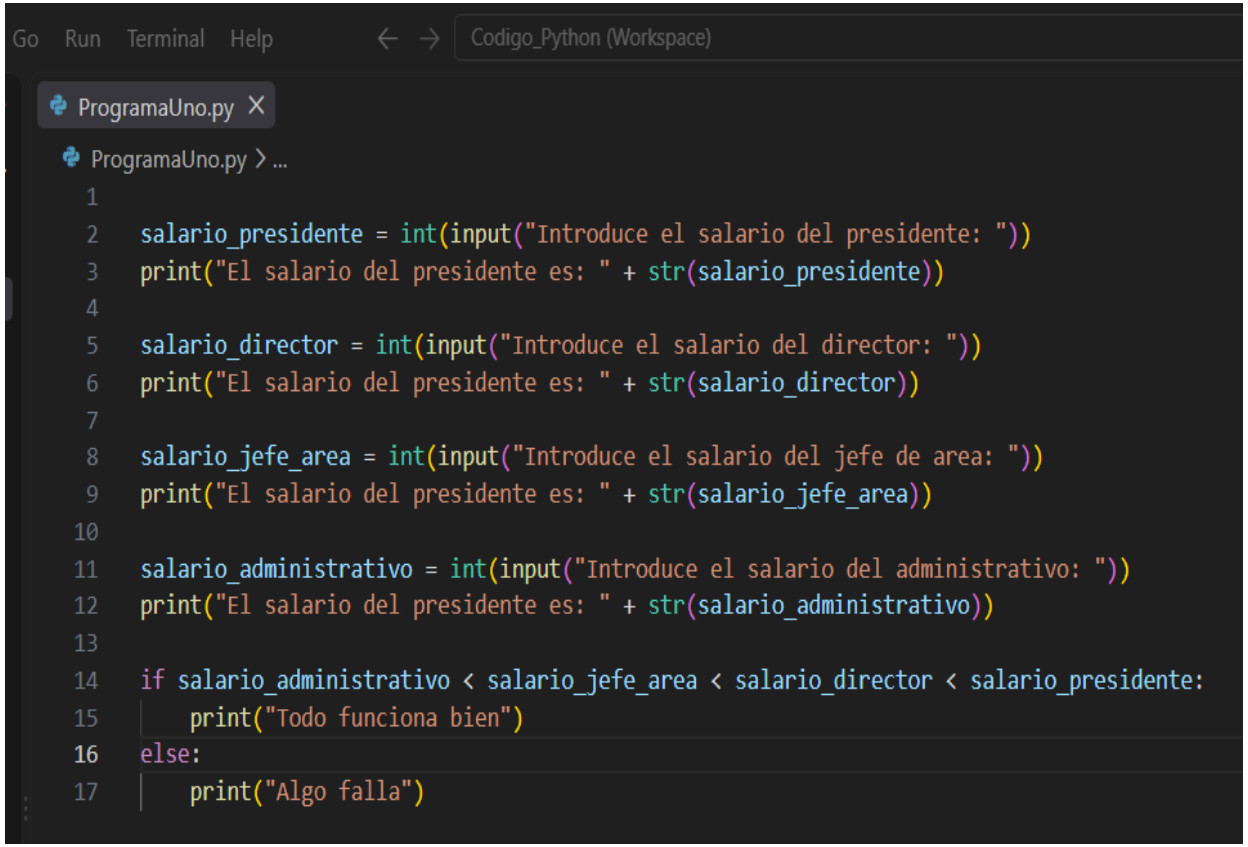

File Edit Selection View Go Run Terminal Help
Codigo_Python (Workspace)
Explorer
Codigo_Python (Workspa...
Codigo_Python
Codigo_Python.code-...
ProgramaUno.py
ProgramaUno.py
ProgramaUno.py > ...
1 # Vamos a trabajar con input para la inserción de datos por teclado
2 print("Programa evaluación de notas")
3
4 nota_alumno = input("Introduce la nota del alumno: ")
5
6 def evaluacion(nota):
7 valoracion = "aprobado"
8 if nota < 5:
9 valoracion = "suspenso"
10 return valoracion
11
12 print(evaluacion(int(nota_alumno)))

EJEMPLO IF ELSE





EJEMPLO


Go Run Terminal Help
Codigo_Python (Workspace)
ProgramaUno.py
ProgramaUno.py > ...
1
2 salario_presidente = int(input("Introduce el salario del presidente: "))
3 print("El salario del presidente es: " + str(salario_presidente))
4
5 salario_director = int(input("Introduce el salario del director: "))
6 print("El salario del presidente es: " + str(salario_director))
7
8 salario_jefe_area = int(input("Introduce el salario del jefe de area: "))
9 print("El salario del presidente es: " + str(salario_jefe_area))
10
11 salario_administrativo = int(input("Introduce el salario del administrativo: "))
12 print("El salario del presidente es: " + str(salario_administrativo))
13
14 if salario_administrativo < salario_jefe_area < salario_director < salario_presidente:
15 print("Todo funciona bien")
16 else:
17 print("Algo falla")

EJEMPLO OPERADORES AND OR

ProgramaUno.py

```python
print("----- Programa de becas -----")
#Parámetros: distancia alumno centro (>40 km), familia numerosa (>2 hermanos) y salario familiar (<=20000 e).

distancia_escuela = int(input("Introduce la distacia a la escuela en km: "))
print(distancia_escuela)

numero_hermanos = int(input("Introduce el número de harmanos en el centro: "))
print(numero_hermanos)

salario_familiar = int(input("Introduce el salario anual bruto: "))
print(salario_familiar)

if distancia_escuela > 40 and numero_hermanos > 2 or salario_familiar <= 20000:
    print("Tienes derecho a beca.")
else:
    print("No tienes derecho a beca.")
```

EJEMPLO OPERADOR IN

ProgramaUno.py

```python
print("----- Asignaturas optativas -----")
# Alumno que cursa una carrera y tiene que elegir una asignatura contemplada en un listado.

print("Optativas: Informática gráfica - Pruebas de software - Usabilidad y accesibilidad.")

opcion = input("Escribe la asignatura a elegir: ")

asignatura = opcion.lower()

if asignatura in ("informática gráfica", "pruebas de software", "usabilidad y accesibilidad"):
    print("Asignatura elegida: " + asignatura)
else:
    print("La asignatura elegida no se contempla.")
```

14. BUCLES

Bucles

Determinados

- Se ejecutan un número determinado de veces.
- Se sabe a priori cuántas veces se va a ejecutar el código del interior del bucle

Indeterminados

- Se ejecutan un número indeterminado de veces
- No se sabe a priori cuántas veces se va a ejecutar el código del interior del bucle.
- El número de veces que se ejecutará dependerá de las circunstancias durante la ejecución del programa

14.1 BUCLES FOR

Bucle for (determinado). Sintaxis

for variable in elemento a recorrer:

Cuerpo del bucle

ProgramaUno.py > ...

```
1
2   for i in [1,2,3]: # Tres elemento, tres repeticiones
3       print("Hola")
```

ProgramaUno.py > ...

```
1
2   for estaciones_anio in ["primavera", "verano", "otoño", "invierno"]: # Nombre muy largo
3       print(estaciones_anio)
```

EJEMPLOS FOR

ProgramaUno.py > ...

```
1
2   for i in [1, 2 ,3 ,4 ]:
3       print("elemento", end=" ") # No hace salto de línea, respeta espacios
```

ProgramaUno.py

ProgramaUno.py > ...

```
1
2   for i in "antonioroman":
3       print("elemento", end=" ") # Lo recorre en función al número de caracteres
4
```

ProgramaUno.py

ProgramaUno.py > ...

```
1   # Dirección de correo electrónico que contenga @, y al menos, un punto.
2   contEmail = 0
3   contPunto = 0
4
5   miEmail = input("Introduce tu dirección de email: ")
6
7   for i in miEmail:
8
9       if (i=="@"):
10          contEmail = contEmail + 1
11      elif (i=="."):
12          contPunto = contPunto + 1
13
14  if (contEmail==1 and (contPunto>=1 and contPunto<=4)):
15      print("El email es correcto")
16  else:
17      print("El email no es correcto")
```

EJEMPLOS FOR

ProgramaUno.py > ...

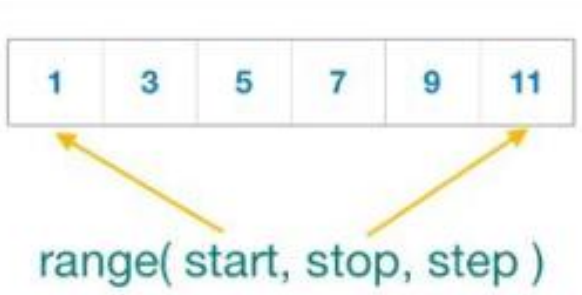
```python
# Nos devuelve una lista de 5 elementos: 0,1,2,3,4
for i in range(5):
    print(i)
```

ProgramaUno.py > ...

```python
# Empieza en 5, termina en 50, avanza de 3 en 3. Probar 2 parámetros.
for i in range(5,50,3):
    print(f"valor de la variable {i}")
```

ProgramaUno.py > ...

```python
# f notación especial. Concatena el texto con el valor de la variable en cada vuelta del bucle
for i in range(5):
    print(f"valor de la variable {i}")
```



ProgramaUno.py

ProgramaUno.py > ...

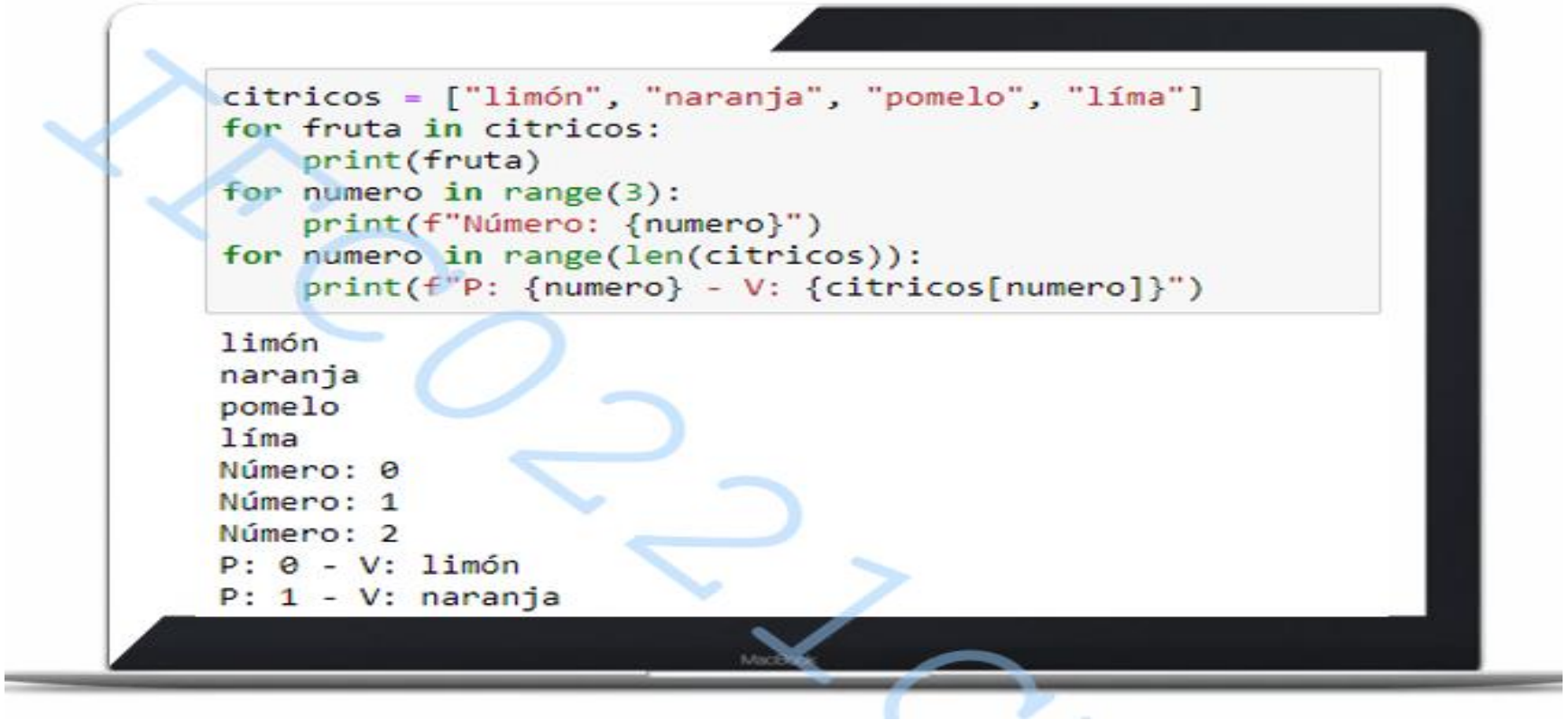
```python

valido =False
email = input("Introduce el email: ")

for i in range(len(email)):
    if email[i] == "@":
        valido = True


if valido:
    print("Email correcto")
else:
    print("Email incorrecto")
```

EJEMPLO FOR


citricos = ["limón", "naranja", "pomelo", "líma"]
for fruta in citricos:
print(fruta)
for numero in range(3):
print(f"Número: {numero}")
for numero in range(len(citricos)):
print(f"P: {numero} - V: {citricos[numero]}")
limón
naranja
pomelo
líma
Número: 0
Número: 1
Número: 2
P: 0 - V: limón
P: 1 - V: naranja

14.2 BUCLES WHILE

En Python tiene una palabra reservada llamada **`while`** que nos permite ejecutar ciclos, o bien secuencias periódicas que nos permiten ejecutar código múltiples veces.

Al igual que la sentencia **`if`**, la estructura **`while`** también puede combinarse con una sentencia **`else`**.

EJEMPLOS WHILE

ProgramaUno.py > ...

```python
# PRimer ejemplo, lo hacemos determinado.
i = 1

while i <= 10:
    print("Ejecución " + str(i))
    i = i + 1

print("Terminó la ejecución")
```

ProgramaUno.py > ...

```python
# PRimer ejemplo, lo hacemos determinado.
edad = int(input("Introduce tu edad: "))

while edad < 5 or edad > 100:
    print("Has introducido una edad erronea. Vuelve a intentarlo.")
    edad = int(input("Introduce tu edad: "))

print("Gracias")
print("Edad del usuario: " + str(edad))
```

ProgramaUno.py > ...

```python
# Programa que averigue la raiz cuadrada de un número. Incluye importar.
import math

print("--- Programa de cálculo de raiz cuadrada ---")
numero = int(input("Introduce un número: "))

intentos = 0

while numero < 0:
    print("No se puede hallar la raiz de un número negativo.")

    if intentos == 2:
        print("Has consumido demasiados intentos")
        print("El programa ha finalizado")
        break; # Con break salimos del bucle

    numero = int(input("Introduce un número: "))

    if numero < 0:
        intentos = intentos + 1

if intentos < 2:
    solucion = math.sqrt(numero) # Clase math
    print("La raiz cuadrada de " + str(numero) + " es: " + str(solucion))
```

EJEMPLO WHILE - ELSE



Python Avanzado

Al igual que la sentencia **if**, la estructura **while** también puede combinarse con una sentencia **else**.

EJEMPLOS WHILE – CONTINUE Y PASS

Continue → Salta a la siguiente iteración del bucle.

Pass → Devuelve null, en cuanto es leído en el interior de un bucle. Un bucle no tiene nada en su interior y pones pass para que el programa se siga ejecutando.

```
for letra in "Phyton":

    if letra == "h":
        continue

    print("Letra: " + letra) # Ignora esta línea de código
```

```
class MiClase:
    pass # Para implementar más tarde, el resto funciona sin problema.
```

ProgramaUno.py > ...

```
nombre = "antonio román"

# print(len(nombre))  Cuenta el espacio en blanco

contador = 0

for i in nombre:
    if i == " ":
        continue
    contador += 1

print(contador)
```

15. GENERADORES

Generadores ¿Qué son?



- Estructuras que extraen valores de una función y se almacenan en objetos iterables (que se pueden recorrer).
- Estos valores se almacenan de uno en uno.
- Cada vez que un generador almacena un valor, esta permanece en un estado pausado hasta que se solicita el siguiente. Esta característica es conocida como “suspensión de estado”.

GENERADORES II



GENERADORES III

Generadores ¿Qué utilidad tienen?

- Son más eficientes que las funciones tradicionales
- Muy útiles con listas de valores infinitos
- Bajo determinados escenarios, será muy útil que un generador devuelva los valores de uno en uno.

EJEMPLO GENERADORES

ProgramaUno.py

ProgramaUno.py > generaPares

```python

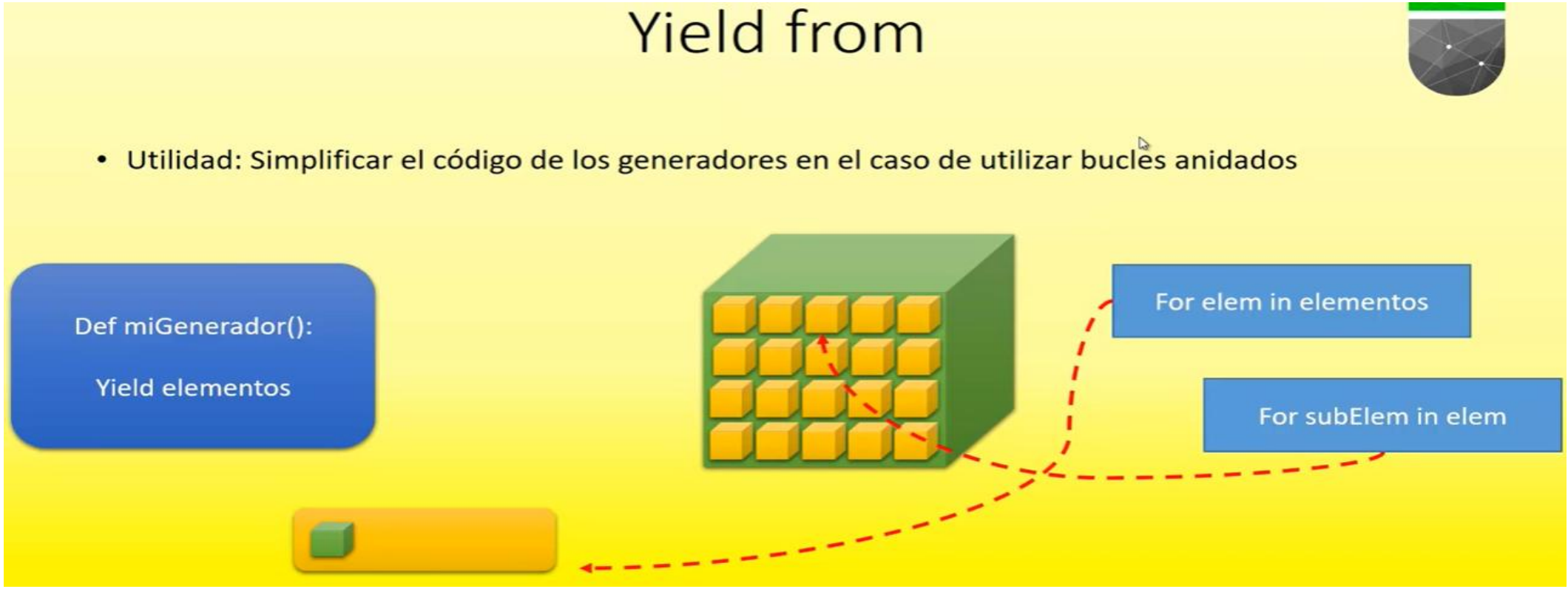
    def generaPares (limite):
        num = 1

        while num < limite:
            yield num*2 # Estado de suspensión, hace que se ahorren recursos.
            num = num + 1

    devuelvePares = generaPares(10)

    print(next(devuelvePares))
    print("------------------")
    print(next(devuelvePares))
    print("------------------")
    print(next(devuelvePares))
```

EJEMPLOS GENERADORES YIELD FROM



EJEMPLO GENERADORES YIELD FROM

ProgramaUno.py > ...

```python

def devuelve_ciudades(*ciudades): # * número indeterminado de elementos en forma de tupla
    for elemento in ciudades:
        #for subElemento in elemento:
            yield from elemento

ciudades_devueltas = devuelve_ciudades("Madrid","Zamora","Valladolid","Cuenca")

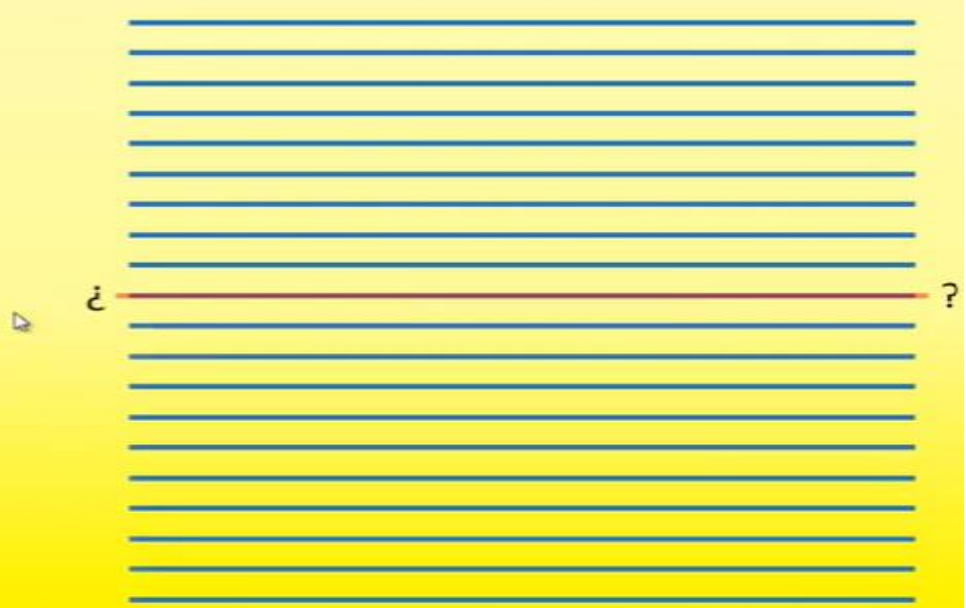
print(next(ciudades_devueltas))
print(next(ciudades_devueltas))

```

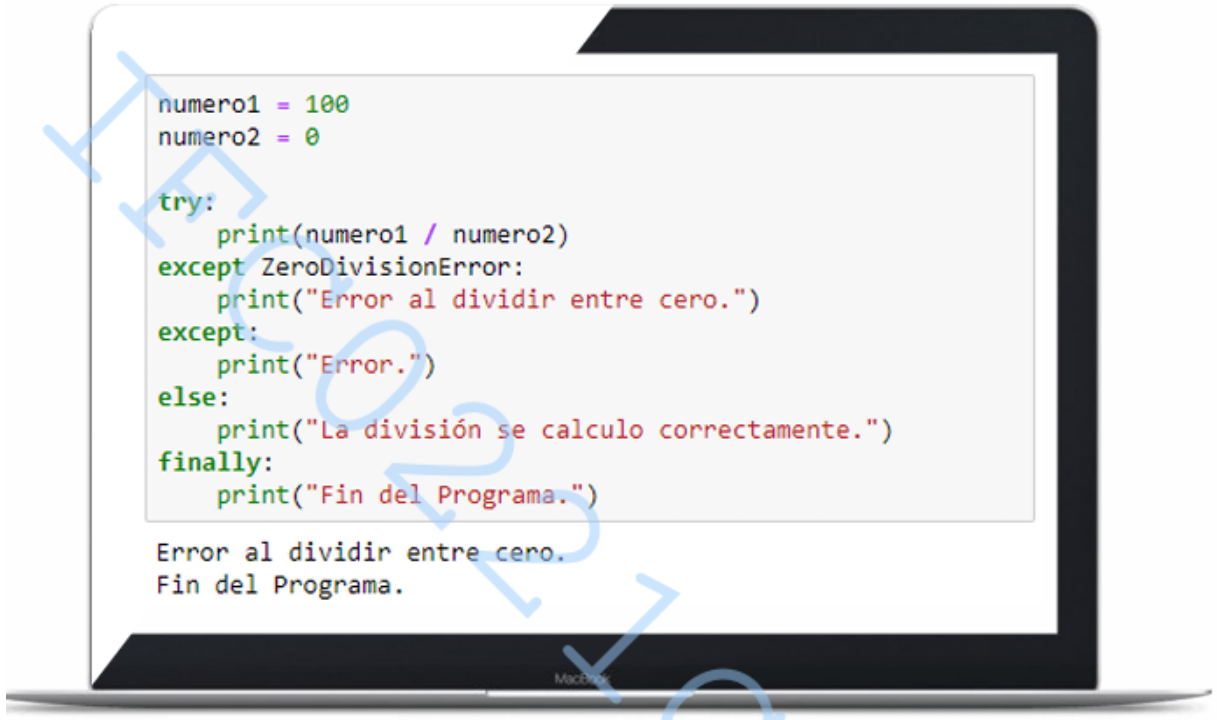
16. EXCEPCIONES

Excepciones ¿Qué son?

- Las excepciones son errores que ocurren durante la ejecución del programa. La sintaxis del código es correcta pero durante la ejecución ha ocurrido “algo inesperado”.
- Este tipo de errores de ejecución se pueden controlar para que la ejecución del programa continúe. Es lo que se conoce como **manejo o control de excepciones.**



EJEMPLO EXCEPCIONES



```
numero1 = 100
numero2 = 0

try:
    print(numero1 / numero2)
except ZeroDivisionError:
    print("Error al dividir entre cero.")
except:
    print("Error.")
else:
    print("La división se calculo correctamente.")
finally:
    print("Fin del Programa.")
```

```
Error al dividir entre cero.
Fin del Programa.
```

CONTROL DE EXCEPCIONES

- **`try`** permite controlar las excepciones producidas en un bloque de código
- **`except`** bloque de instrucciones que se ejecutan cuando se produce una excepción
- **`else`** bloque de instrucciones que se ejecutan al finalizar el *try* si no se produce un excepción
- **`finally`** bloque de instrucciones que se ejecutan siempre que finaliza el *try, except* o *else*

EJEMPLO EXCEPCIONES PASO A PASO

prueba_excepciones.py > ...

```python
def suma(num1, num2):
    return num1+num2

def resta(num1, num2):
    return num1-num2

def multiplica(num1, num2):
    return num1*num2

def divide(num1,num2):
    return num1/num2


op1=(int(input("Introduce el primer número: ")))

op2=(int(input("Introduce el segundo número: ")))

operacion=input("Introduce la operación a realizar (suma,resta,multiplica,divide): ")

if operacion=="suma":
    print(suma(op1,op2))

elif operacion=="resta":
    print(resta(op1,op2))

elif operacion=="multiplica":
    print(multiplica(op1,op2))

elif operacion=="divide":
    print(divide(op1,op2))

else:
    print ("Operación no contemplada")


print("Operación ejecutada. Continuación de ejecución del programa ")
```

```
PS C:\Users\anton\Desktop\Codigo_Python> & C:\Users\anton\.local\bin\python3.14.exe c:/Users/anton/Desktop/Codigo_Python/prueba_excepciones.py
Introduce el primer número: 8
Introduce el segundo número: 0
Introduce la operación a realizar (suma,resta,multiplica,divide): divide
Traceback (most recent call last):
  File "c:\Users\anton\Desktop\Codigo_Python\prueba_excepciones.py", line 30, in <module>
    print(divide(op1,op2))
          ~~~~~~^^^^^^^^^
  File "c:\Users\anton\Desktop\Codigo_Python\prueba_excepciones.py", line 11, in divide
    return num1/num2
           ~~~~^~~~
ZeroDivisionError: division by zero
```

EJEMPLO EXCEPCIONES PASO A PASO II

```
7   def multiplica(num1, num2):
8       return num1*num2
9
10  def divide(num1,num2):
11      try:
12          return num1/num2
13      except ZeroDivisionError:
14          print("No se puede dividir entre cero.")
15          return "Operación Erronea"
16
17
18  op1=(int(input("Introduce el primer número: ")))
19
20  op2=(int(input("Introduce el segundo número: ")))
```

```
Introduce el primer número: 8
Introduce el segundo número: 0
Introduce la operación a realizar (suma,resta,multiplica,divide): divide
No se puede dividir entre cero.
Operación Erronea
Operación ejecutada. Continuación de ejecución del programa
PS C:\Users\anton\Desktop\Codigo_Python>
```

EJEMPLO EXCEPCIONES PASO A PASO III

```
Introduce el primer número: sdfsdfasdf
Traceback (most recent call last):
  File "c:\Users\anton\Desktop\Codigo_Python\prueba_excepciones.py", line 18, in <module>
    op1=(int(input("Introduce el primer número: ")))
         ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'sdfsdfasdf'
```

```
Introduce el primer número: asdasdd
Los valores introducidos no son correctos.
Introduce la operación a realizar (suma,resta,multiplica,divide): suma
Traceback (most recent call last):
  File "c:\Users\anton\Desktop\Codigo_Python\prueba_excepciones.py", line 28, in <module>
    print(suma(op1,op2))
              ^^^
NameError: name 'op1' is not defined
```

```
17  while True:
18      try:
19          op1=(int(input("Introduce el primer número: ")))
20          op2=(int(input("Introduce el segundo número: ")))
21
22          break
23
24      except ValueError:
25          print("Los valores introducidos no son correctos. Inténtalo de nuevo.")
26
27  operacion=input("Introduce la operación a realizar (suma,resta,multiplica,divide): ")
```

EJEMPLO CAPTURA DE EXCEPCIONES CONSECUTIVA

ProgramaUno.py > divide

```
1
2     def divide():
3         try:
4             op1 = (float(input("Introduce el primer número: ")))
5             op2 = (float(input("Introduce el segundo número: ")))
6
7             print("La división es: " + str(op1/op2))
8         except ValueError:
9             print("El valor introducido es erroneo.")
10        except ZeroDivisionError:
11            print("No es posible dividir entre cero.")
12        finally:
13            print("Cálculo finalizado")
14
15    divide()
```

EJEMPLO PERSONALIZAR MENSAJE ERROR

```
2   def evaluaEdad(edad):
3       if edad < 0:
4           raise TypeError("No se permiten edades negativas") #Personalizar el mensaje
5       if edad < 20:
6           print("Eres muy joven")
7       elif edad < 40:
8           print("Eres joven")
9       elif edad < 65:
10          print("Eres mayor")
11      elif edad < 100:
12          print("Cuídate...")
13
14  evaluaEdad(-15)
```

EJEMPLO

ProgramaUno.py > ...

```python
import math
def calculaRaiz(num1):
    if num1 < 0:
        raise ValueError("El número no puede ser negativo")
    else:
        return math.sqrt(num1)

op1 = (int(input("Introduce un número: ")))

try:
    print(calculaRaiz(op1))
except ValueError as ErrorDeNumeroNegativo:
    print(ErrorDeNumeroNegativo) # Nos saca el mensaje contenido en ValueError

print("Programa terminado.")
```

17. POO

- ¿En qué consiste?
  - Trasladar la naturaleza de los objetos de la vida real al código de programación.
- ¿Cuál es la naturaleza de un objeto de la vida real?
  - Los objetos tienen un estado, un comportamiento (¿Qué puede hacer?), y unas propiedades
- Pongamos un ejemplo: El objeto coche.
  - ¿Cuál es el estado de un coche? Un coche puede estar parado, circulando, aparcado etc
  - ¿Qué propiedades tiene un coche? Un coche tiene un color, un peso, un tamaño etc.
  - ¿Qué comportamiento tiene un coche? Un coche puede arrancar, frenar, acelerar, girar etc.

POO II

- Objeto:
  - Tiene propiedades (atributos):
    - Color
    - Peso
    - Alto
    - Largo
  - Tiene un comportamiento (¿Qué es capaz de hacer?):
    - Arrancar
    - Frenar
    - Girar
    - Acelerar

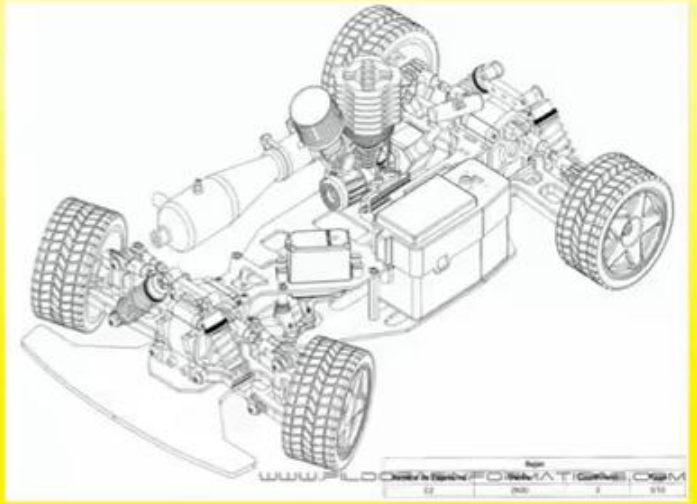




POO III

- Programación Orientada a objetos:
  - Algunos ejemplos de lenguajes: C++, Java, Visual.NET etc.
  - **Ventajas**:
    - Programas divididos en “trozos”, “partes”, “módulos” o “clases”. Modularización.
    - Muy reutilizable. Herencia.
    - Si existe fallo en alguna línea del código, el programa continuará con su funcionamiento. Tratamiento de Excepciones.
    - Encapsulamiento.





POO IV

Clase: Plantilla o modelo que define las características (atributos) y comportamientos (métodos) que tendrán sus objetos.

Objeto: Elemento creado a partir de una clase que contiene sus propios datos y puede ejecutar sus métodos.

Instancia de clase: Objeto concreto que ha sido creado a partir de una determinada clase.

Modularización: División de un programa en módulos independientes y organizados, facilitando su desarrollo, mantenimiento y reutilización.

Encapsulación: Mecanismo que oculta los datos internos de un objeto y controla el acceso a ellos mediante métodos o interfaces.

Herencia: Mecanismo por el que una clase puede adquirir atributos y métodos de otra clase, permitiendo reutilizar y ampliar código.

Polimorfismo: Capacidad de un mismo método o interfaz de comportarse de distintas formas dependiendo del objeto que lo utilice.

POO V

Objeto.

Accediendo a propiedades y comportamiento (pseudocódigo)

- Objeto:
  - Accediendo a propiedades de objeto desde código (pseudocódigo):
    - miCoche.color="rojo";
    - miCoche.peso=1500;
    - miCoche.ancho=2000;
    - miCoche.alto=900;
  - Accediendo a comportamiento de objeto desde código (pseudocódigo):
    - miCoche.arranca();
    - miCoche.frena();
    - miCoche.gira();
    - miCoche.acelera();

EJEMPLO POO

ProgramaUno.py > Coche > estado

```python
class Coche():
    #Propiedades
    largoChasis=250
    anchoChasis=120
    ruedas=4
    enmarcha=False
    #Métodos
    def arrancar(self): #self es el this de java
        self.enmarcha=True

    def estado(self):
        if(self.enmarcha):
            return "El coche está en marcha"
        else:
            return "El coche está parado"

miCoche=Coche() #Instanciar
print("El largo del coche es: ", miCoche.largoChasis)
print("El coche tiene: ", miCoche.ruedas, "ruedas.")
miCoche.arrancar()

print(miCoche.estado())
```

EJEMPLO PARÁMETRO

ProgramaUno.py > ...

```python
class Coche():
    #Propiedades
    largoChasis=250
    anchoChasis=120
    ruedas=4
    enmarcha=False
    #Métodos
    def arrancar(self, arrancamos): #self es el this de java. Parámetro.
        self.enmarcha=arrancamos
        if(self.enmarcha):
            return "El coche está en marcha"
        else:
            return "El coche está parado"

    def estado(self):
        print("El coche tiene", self.ruedas, " ruedas. Un ancho de", self.anchoChasis,
            " y un largo de ", self.largoChasis)

```

```python

miCoche=Coche() #Instanciar
print(miCoche.arrancar(True))
miCoche.estado()

print("------------ A continuación creamos el segundo objeto --------------")

miCoche2=Coche()
print(miCoche.arrancar(False))
miCoche2.estado()
```

EJEMPLO CONSTRUCTOR

ProgramaUno.py > ...

```python
class Coche():
    #Constructor
    def __init__(self): #Estado inicial
        #Propiedades
        self.__largoChasis=250
        self.__anchoChasis=120
        self.__ruedas=4 # Para encapsular dos __
        self.__enmarcha=False

    #Métodos
    def arrancar(self, arrancamos): #self es el this de java. Parámetro.
        self.__enmarcha=arrancamos
        if(self.__enmarcha):
            return "El coche está en marcha"
        else:
            return "El coche está parado"

    def estado(self):
        print("El coche tiene", self.__ruedas, " ruedas. Un ancho de", self.__anchoChasis,
            " y un largo de ", self.__largoChasis)
```

```python

miCoche=Coche() #Instanciar
print(miCoche.arrancar(True))
miCoche.estado()

print("------------ A continuación creamos el segundo objeto -------------")

miCoche2=Coche()
print(miCoche.arrancar(False))
miCoche2.ruedas = 2 # importante encapsular, proteger una propiedad para que no se
# modifique desde fuera. Probar con miCoche2.__ruedas , da igual
miCoche2.estado()
```

EJEMPLO ENCAPSULAR UN MÉTODO

ProgramaUno.py > ...

```python
class Coche():
    #Constructor
    def __init__(self): #Estado inicial
        #Propiedades
        self.__largoChasis=250
        self.__anchoChasis=120
        self.__ruedas=4 # Para encapsular dos __
        self.__enmarcha=False

    #Métodos
    def arrancar(self, arrancamos): #self es el this de java. Parámetro.
        self.__enmarcha=arrancamos

        if(self.__enmarcha):
            chequeo=self.__chequeo_interno() #Guardamos el true o el false

        if(self.__enmarcha and chequeo):
            return "El coche está en marcha"
        elif(self.__enmarcha and chequeo==False):
            return "Algo ha ido mal en el chequeo. No se puede arrancar"
        else:
            return "El coche está parado"

    def estado(self):
        print("El coche tiene", self.__ruedas, " ruedas. Un ancho de", self.__anchoChasis,
            " y un largo de ", self.__largoChasis)

    #Método que haga un chequeo antes de arrancar. Gasolina, aceite... Ok
    def __chequeo_interno(self):
        print("Realizando chequeo interno")
        #Inicialmente todo ok.
        self.gasolina="ok"
        self.aceite="ok" # Cambiar a "mal" y comprobar
        self.puertas="cerradas"
        if(self.gasolina=="ok" and self.aceite=="ok" and self.puertas=="cerradas"):
            return True
        else:
            return False



print("------------ A continuación creamos el primer objeto -------------")
miCoche=Coche() #Instanciar
print(miCoche.arrancar(True))
miCoche.estado()
# print(miCoche.__chequeo_interno()) No tendría sentido, para ello encapsular método

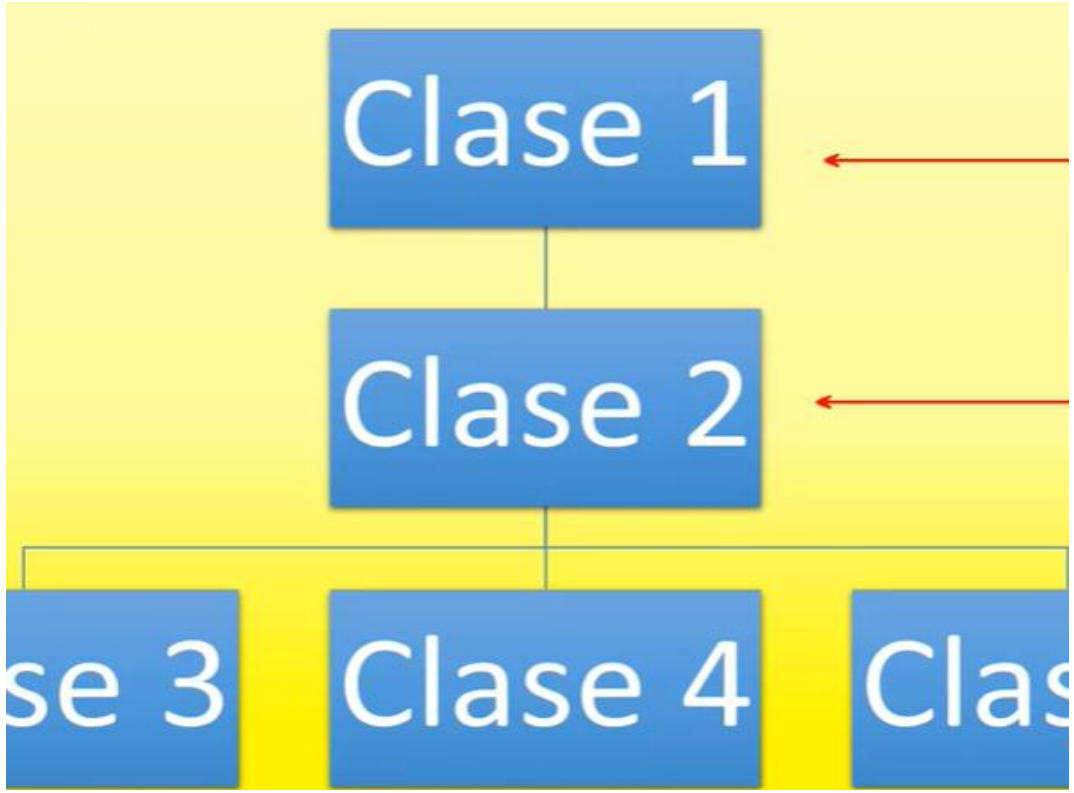
print("------------ A continuación creamos el segundo objeto -------------")

miCoche2=Coche()
print(miCoche.arrancar(False))
miCoche2.estado()
# print(miCoche2.__chequeo_interno()) No tendría sentido, para ello encapsular método
```

17.1 POO HERENCIA

La **herencia** en Python es un mecanismo de la **programación orientada a objetos (POO)** que permite crear una clase nueva a partir de otra clase existente.

La idea principal es que una clase hija puede **heredar atributos y métodos** de una clase padre, y además puede añadir o modificar su propio comportamiento.



17.1 POO HERENCIA II

¿Para qué sirve la herencia?

- Para reutilizar código en caso de crear objetos similares



¿Qué características en común tienen todos los objetos?

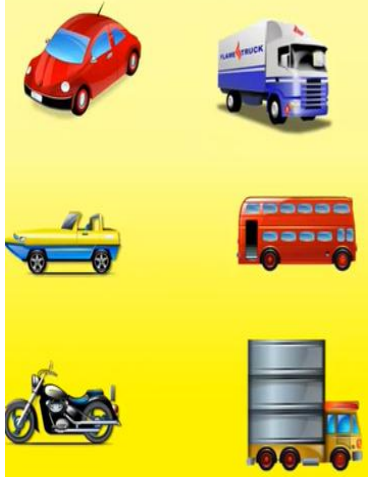
Marca | Modelo

¿Qué comportamientos en común tienen todos los objetos?

Arrancan | Aceleran | Frenan

¿Para qué sirve la herencia?

- Para reutilizar código en caso de crear objetos similares



CLASE PADRE O SUPERCLASE

Marca | Modelo

Arrancan | Aceleran | Frenan

EJEMPLO HERENCIA

```python
class Vehiculos():
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo
        self.enmarcha = False
        self.acelera = False
        self.frena = False

    def arrancar(self):
        self.enmarcha = True

    def acelerar(self):
        self.acelera = True

    def frenar(self):
        self.frena = True

    def estado(self):
        print("Marca: ", self.marca, "\nModelo: ", self.modelo, "\nEn Marcha: ",
        self.enmarcha, "\nAcelerando: ", self.acelera, "\nFrenando: ", self.frena,)

class Moto(Vehiculos):
    pass

miMoto = Moto("Honda", "CBR")
miMoto.estado()
```

EJEMPLO HERENCIA MULTIPLE

ProgramaUno.py > ...

```python
class Vehiculos():
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo
        self.enmarcha = False
        self.acelera = False
        self.frena = False

    def arrancar(self):
        self.enmarcha = True

    def acelerar(self):
        self.acelera = True

    def frenar(self):
        self.frena = True

    def estado(self):
        print("Marca: ", self.marca, "\nModelo: ", self.modelo, "\nEn Marcha: ",
        self.enmarcha, "\nAcelerando: ", self.acelera, "\nFrenando: ", self.frena,)

class Furgoneta(Vehiculos):
    def carga(self, cargar):
        self.cargado=cargar
        if(self.cargado):
            return "La furgoneta está cargada"
        else:
            return "La furgoneta no está cargada"
```

```python
class Moto(Vehiculos):
    hcaballito=""
    def caballito(self):
        self.hcaballito="Voy haciendo un caballito"

    def estado(self):
            print("Marca: ", self.marca, "\nModelo: ", self.modelo, "\nEn Marcha: ",
            self.enmarcha, "\nAcelerando: ", self.acelera, "\nFrenando: ", self.frena, "\n", self.hcaballito)

class VElectricos():
    def __init__(self):
        self.autonomia=100

    def cargarEnergia(self):
        self.cargando=True

miMoto = Moto("Honda", "CBR")
miMoto.caballito()
miMoto.estado() # Llamamos a estado perteneciente a moto

miFurgoneta = Furgoneta("Renault", "Kangoo")
miFurgoneta.arrancar()
miFurgoneta.estado()
print(miFurgoneta.carga(True))
```

EJEMPLO HERENCIA MULTIPLE II

```
54
55  class BicicletaElectrica(VElectricos, Vehiculos): # Herencia multiple. Se le da prioridad al constructor del primerc
56      pass
57
58  miBici=BicicletaElectrica()
```

EJEMPLO SUPER ISINSTANCE

ProgramaDos.py > Empleado > descripcion

```python
class Persona():
    def __init__(self, nombre, edad, Lugar_residencia):
        self.nombre = nombre
        self.edad = edad
        self.lugar_residencia = Lugar_residencia

    def descripcion(self):
        print("Nombre: ", self.nombre, "Edad: ", self.edad, "Residencia: ", self.lugar_residencia)

class Empleado(Persona):
    def __init__(self, salario, antiguedad, nombre_empleado, edad_empleado, residencia_empleado):
        # Llamamos al método de la clase padre
        super().__init__(nombre_empleado, edad_empleado, residencia_empleado)
        self.salario = salario
        self.antiguedad = antiguedad

    def descripcion(self):
        super().descripcion()
        print("Salario: ", self.salario, "Antiguedad: ", self.antiguedad)

print("----------------------")
Antonio = Empleado(1500, 15, "Antonio", 55, "España")
Antonio.descripcion()
print("----------------------")
Manuel = Persona("Manuel", 60, "España")
Manuel.descripcion()
print("----------------------")
print(isinstance(Antonio, Empleado))
print(isinstance(Antonio, Persona))
print(isinstance(Manuel, Persona))
print(isinstance(Manuel, Empleado))
```

17.2 POLIMORFISMO

Polimorfismo.py > ...

```python
class Coche():
    def desplazamiento(self):
        print("Me desplazo utilizando 4 ruedas")

class Moto():
    def desplazamiento(self):
        print("Me desplazo utilizando 2 ruedas")

class Camion():
    def desplazamiento(self):
        print("Me desplazo utilizando 6 ruedas")


def desplazamientoVehiculo(vehiculo):
    vehiculo.desplazamiento() # Elige el desplazamiento de camión automáticamente

#miVehiculo = Moto()
#miVehiculo.desplazamiento()

#miVehiculo2 = Coche()
#miVehiculo2.desplazamiento()

#miVehiculo3 = Camion()
#miVehiculo3.desplazamiento()

miVehiculo = Camion() # Cambiar por Coche y Moto
desplazamientoVehiculo(miVehiculo)
```

FIN